MySQL

Complete Mastery Guide

PART 2 — Advanced Concepts

String Functions · Numeric Functions · Date/Time Functions · Subqueries · Window Functions · CTEs · Views · Triggers · Stored Procedures · Transactions · Indexes · Normalization · Error Handling · Optimization · Interview Q&A · Coding Challenges

★ Before & After Tables ★ Multi-Approach Solutions ★ Optimisation Analysis ★

Part 2 — Table of Contents

1. String Functions — Complete Reference (20+ functions)
2. Numeric Functions — Complete Reference
3. Date & Time Functions — Complete Reference (20+ functions)
4. Subqueries — All Types (Scalar, Multi-row, Correlated, Derived)
5. Window Functions — ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST/LAST VALUE, Running Totals
6. Common Table Expressions (CTEs) — Simple, Multiple, Recursive
7. Views — CREATE, Updatable Views, WITH CHECK OPTION
8. Triggers — BEFORE/AFTER INSERT/UPDATE/DELETE
9. Stored Procedures — Parameters, IF/ELSE, LOOP, CURSOR
10. Stored Functions — User-Defined Functions
11. Transactions & ACID — COMMIT, ROLLBACK, SAVEPOINT, Isolation Levels
12. Indexes & EXPLAIN — B-Tree, FULLTEXT, Composite, Query Analysis
13. Normalization — 1NF, 2NF, 3NF, BCNF
14. Error Handling — SIGNAL, DECLARE HANDLER
15. Performance Optimization — 15 Essential Rules
16. MySQL AUTO_INCREMENT Advanced & Date Handling

SECTION A: 30 Master Interview Questions & Answers

SECTION B: 15 Coding Challenges with Multi-Approach Solutions

1. String Functions — Complete Reference

⍰ Interview-Ready Definition

String functions transform, extract, search, and format text data. MySQL provides 30+ string functions. Essential for data cleaning, ETL pipelines, report formatting, and text processing.

Complete String Functions Reference Table

Function	Purpose	Syntax Example	Result
UPPER(str)	Convert to UPPERCASE	UPPER('hello')	HELLO
LOWER(str)	Convert to lowercase	LOWER('MYSQL')	mysql
LENGTH(str)	Byte length of string	LENGTH('Hyderabad')	9
CHAR_LENGTH(str)	Character count (Unicode-safe)	CHAR_LENGTH('□□□□□□')	6
CONCAT(a,b,...)	Join strings together	CONCAT('Hello',' ','World')	Hello World
CONCAT_WS(sep,...)	Join with separator (skips NULL)	CONCAT_WS('-','2024','01')	2024-01
SUBSTRING(s,pos,len)	Extract from position	SUBSTRING('MySQL',2,3)	ySQ
SUBSTR(s,pos,len)	Alias of SUBSTRING	SUBSTR('Hello',1,3)	Hel
LEFT(str,n)	First n characters	LEFT('Hyderabad',3)	Hyd
RIGHT(str,n)	Last n characters	RIGHT('Hyderabad',3)	bad
TRIM(str)	Remove leading & trailing spaces	TRIM(' hello ')	hello
LTRIM(str)	Remove leading spaces only	LTRIM(' hello')	hello
RTRIM(str)	Remove trailing spaces only	RTRIM('hello ')	hello
REPLACE(s,old,new)	Replace all occurrences	REPLACE('cat','c','b')	bat
INSTR(str,sub)	Position of first occurrence	INSTR('MySQL','SQL')	3
LOCATE(sub,str)	Same as INSTR (ANSI standard)	LOCATE('SQL','MySQL')	3
LPAD(str,len,pad)	Pad string on left to length	LPAD('5',5,'0')	00005
RPAD(str,len,pad)	Pad string on right to length	RPAD('Hi',5,'!')	Hi!!!
REVERSE(str)	Reverse the string	REVERSE('MySQL')	LQSyM
REPEAT(str,n)	Repeat string n times	REPEAT('ab',3)	ababab
SPACE(n)	Return n spaces	SPACE(5)	(5 spaces)
FORMAT(n,d)	Format number with commas	FORMAT(1234567.89,2)	1,234,567.89

Function	Purpose	Syntax Example	Result
ASCII(str)	ASCII code of first character	ASCII('A')	65
CHAR(n)	Character from ASCII code	CHAR(65)	A
ORD(str)	ASCII/Unicode of first char	ORD('A')	65
HEX(str)	Hexadecimal representation	HEX('A')	41
UNHEX(hex)	Convert hex back to string	UNHEX('41')	A
MD5(str)	32-char MD5 hash (lowercase)	MD5('password')	5f4d...
SHA1(str)	40-char SHA1 hash	SHA1('password')	c3499c...
STRCMP(a,b)	Compare strings: -1/0/1	STRCMP('abc','abd')	-1
SOUNDEX(str)	Phonetic code of string	SOUNDEX('Ravi')	R100
REGEXP_REPLACE(s,pattern,repl)	Replace regex match	REGEXP_REPLACE('abc123','[0-9]','#')	abc###

Example 1: UPPER, LOWER, LENGTH, CHAR_LENGTH

▶ students table — full_name and email columns

student_id	full_name	email	city
1	Amit Joshi	amit@gmail.com	Hyderabad
2	Meera Nair	meera@gmail.com	Kochi
7	Deepak Verma	deepak@gmail.com	Pune

```
SELECT
  full_name,
  UPPER(full_name)                AS name_upper,
  LOWER(email)                    AS email_lower,
  LENGTH(full_name)               AS byte_length,
  CHAR_LENGTH(full_name)          AS char_count,
  LENGTH(email) - CHAR_LENGTH(email) AS extra_bytes   -- non-ASCII chars = >0
FROM students
ORDER BY full_name LIMIT 4;
```

Output:

```
full_name     | name_upper    | email_lower        | byte_length | char_count | extra_bytes
--------------+---------------+--------------------+-------------+------------+------------
Amit Joshi    | AMIT JOSHI    | amit@gmail.com     | 10          | 10         | 0
Anjali Singh  | ANJALI SINGH  | anjali@gmail.com   | 12          | 12         | 0
Deepak Verma  | DEEPAK VERMA  | deepak@gmail.com   | 12          | 12         | 0
Kavya Reddy   | KAVYA REDDY   | kavya@gmail.com    | 11          | 11         | 0
```

How it works: LENGTH() returns bytes. CHAR_LENGTH() returns characters. For ASCII text they are equal. For UTF-8 Hindi/emoji characters, one character = 3-4 bytes, so LENGTH > CHAR_LENGTH.

Example 2: CONCAT and CONCAT_WS — Build Formatted Strings

```
SELECT
  full_name,
  email,
  -- CONCAT: joins all args (NULL in any arg → whole result is NULL)
  CONCAT(full_name, ' <', email, '>') AS formatted_contact,
  -- CONCAT_WS: skips NULL args, uses separator between non-NULLs
  CONCAT_WS(' | ', full_name, city, phone) AS info_line,
  CONCAT_WS('-', YEAR(joined_date), MONTH(joined_date), DAY(joined_date)) AS join_yyyymd
FROM students
ORDER BY full_name LIMIT 4;
```

Output:

```
full_name      | formatted_contact                  | info_line                              |
join_yyyymd
---------------+------------------------------------+----------------------------------------+-----
------
Amit Joshi     | Amit Joshi <amit@gmail.com>        | Amit Joshi | Hyderabad | 9000000001    |
2022-1-10
Anjali Singh   | Anjali Singh <anjali@gmail.com>    | Anjali Singh | Mumbai | 9000000004    | 2023-
2-5
Deepak Verma   | Deepak Verma <deepak@gmail.com>    | Deepak Verma | Pune               | 2021-8-25
Kavya Reddy    | Kavya Reddy <kavya@gmail.com>      | Kavya Reddy | Hyderabad | 9000000006 | 2023-
1-1
```

How it works: CONCAT_WS skips NULL values automatically — Deepak's phone is NULL, so the line shows only name and city without a dangling separator. CONCAT would return NULL for the whole field if phone is NULL.

Example 3: SUBSTRING, LEFT, RIGHT, INSTR — Extract Parts of Strings

```
SELECT
  email,
  -- Extract username (before @)
  LEFT(email, INSTR(email,'@') - 1)                    AS username,
  -- Extract domain (after @)
  SUBSTRING(email, INSTR(email,'@') + 1)               AS domain,
  -- First 3 chars of city
  LEFT(city, 3)                                        AS city_code,
  -- Last 4 chars of phone (mask the rest)
  CONCAT('******', RIGHT(phone, 4))                    AS masked_phone
FROM students
WHERE phone IS NOT NULL
ORDER BY full_name LIMIT 4;
```

Output:

```
email               | username | domain      | city_code | masked_phone
--------------------+----------+-------------+-----------+-------------
amit@gmail.com      | amit     | gmail.com   | Hyd       | ******0001
anjali@gmail.com    | anjali   | gmail.com   | Mum       | ******0004
kavya@gmail.com     | kavya    | gmail.com   | Hyd       | ******0006
meera@gmail.com     | meera    | gmail.com   | Koc       | ******0002
```

How it works: INSTR(email,'@') returns position of '@'. LEFT(..., pos-1) extracts everything before it. SUBSTRING(..., pos+1) extracts everything after. Combining these extracts email parts dynamically.

Example 4: TRIM, LTRIM, RTRIM, REPLACE — Data Cleaning

▶ Dirty data before cleaning

raw_name	raw_city
Amit Joshi	HYDERABAD
Meera Nair	bengaluru

raw_name	raw_city
Rahul Gupta	DELHI

```
SELECT
  raw_name,
  TRIM(raw_name)                    AS trimmed_name,
  TRIM(raw_city)                    AS trimmed_city,
  -- Standardise city names
  REPLACE(REPLACE(UPPER(TRIM(raw_city)),
    'BENGALURU','BANGALORE'),
    'BOMBAY','MUMBAI')              AS std_city,
  -- Remove internal double spaces
  REPLACE(TRIM(raw_name),'  ',' ') AS clean_name
FROM dirty_contacts;
```

Output:

```
raw_name       | trimmed_name | trimmed_city | std_city   | clean_name
---------------+--------------+--------------+-----------+-----------
  Amit Joshi   | Amit Joshi   | HYDERABAD    | HYDERABAD | Amit Joshi
Meera  Nair    | Meera  Nair  | bengaluru    | BANGALORE | Meera Nair
  Rahul Gupta  | Rahul Gupta  | DELHI        | DELHI     | Rahul Gupta
```

How it works: TRIM removes leading/trailing spaces. REPLACE(str,old,new) replaces ALL occurrences. Nesting REPLACE calls handles multiple substitutions. This is the standard ETL data cleaning pattern.

Example 5: LPAD, RPAD — Formatting and Padding

```
SELECT
  student_id,
  -- Format ID as STU000001 (padded to 6 digits)
  CONCAT('STU', LPAD(student_id, 6, '0'))       AS student_code,
  -- Format order number like ORD-000001
  CONCAT('ORD-', LPAD(order_id, 6, '0'))        AS order_number,
  -- Right-pad a status to fixed width for display
  RPAD(status, 12, '.')                         AS status_display
FROM orders
LIMIT 4;
```

Output:

```
student_id | student_code | order_number  | status_display
-----------+--------------+---------------+---------------
1          | STU000001    | ORD-000001    | completed...
1          | STU000001    | ORD-000002    | completed...
2          | STU000002    | ORD-000003    | completed...
2          | STU000002    | ORD-000004    | completed...
```

How it works: LPAD(student_id, 6, '0') pads the ID to 6 characters with leading zeros. Essential for generating invoice numbers, order codes, and formatted ID systems.

Example 6: FORMAT, REVERSE, REPEAT, SOUNDEX

```
SELECT
  price,
  FORMAT(price, 2)                  AS formatted_price,   -- 1,499.00
  FORMAT(price, 0)                  AS no_decimals,        -- 1,499
  REVERSE('MySQL')                  AS reversed,           -- LQSyM
  REPEAT('★', 5)                    AS five_stars,
  SOUNDEX('Ravi')                   AS soundex_ravi,       -- R100
  SOUNDEX('Ravi')  = SOUNDEX('Ravy') AS sounds_same        -- 1 (TRUE)
FROM courses LIMIT 3;
```

Output:

```
price   | formatted_price | no_decimals | reversed | five_stars | soundex_ravi | sounds_same
--------+-----------------+-------------+----------+------------+--------------+------------
1499.00 | 1,499.00        | 1,499       | LQSyM    | ★★★★★      | R100         | 1
2999.00 | 2,999.00        | 2,999       | LQSyM    | ★★★★★      | R100         | 1
1999.00 | 1,999.00        | 1,999       | LQSyM    | ★★★★★      | R100         | 1
```

Example 7: REGEXP_REPLACE — Advanced Pattern Replacement

```
-- Mask all digits in a phone number
SELECT
  phone,
  REGEXP_REPLACE(phone, '[0-9]', '*')          AS masked_phone,
  -- Remove all non-alphanumeric from a string
  REGEXP_REPLACE('Hello, World! 123', '[^a-zA-Z0-9]', '') AS alphanum_only,
  -- Replace multiple spaces with single space
  REGEXP_REPLACE('Amit   Joshi   Ravi', '\\s+', ' ')    AS normalised
FROM students LIMIT 3;
```

Output:

```
phone      | masked_phone | alphanum_only   | normalised
-----------+--------------+-----------------+---------------
9000000001 | **********   | HelloWorld123   | Amit Joshi Ravi
```

❓ What is the difference between LENGTH() and CHAR_LENGTH()?

✅LENGTH() returns the BYTE count of a string. For ASCII characters they are equal. For multi-byte character sets (UTF-8), a single character can be 1-4 bytes. CHAR_LENGTH() always returns the CHARACTER count regardless of encoding. Use CHAR_LENGTH() for Unicode-safe code.

❓ What is the difference between CONCAT() and CONCAT_WS()?

✅CONCAT(a,b,c) joins arguments directly — if ANY argument is NULL, the entire result is NULL. CONCAT_WS(sep, a, b, c) joins with a separator and SKIPS NULL arguments automatically. Use CONCAT_WS when columns may be NULL.

2. Numeric Functions — Complete Reference

Interview-Ready Definition

Numeric functions perform mathematical operations on numbers — rounding, absolute values, power, square root, trigonometry, and random number generation.

Numeric Functions Reference

Function	Purpose	Example	Result
ROUND(n,d)	Round to d decimal places	ROUND(1234.5678, 2)	1234.57
ROUND(n)	Round to nearest integer	ROUND(4.5)	5
FLOOR(n)	Round DOWN to nearest int	FLOOR(4.9)	4
CEILING(n)	Round UP to nearest int	CEILING(4.1)	5
CEIL(n)	Alias for CEILING	CEIL(4.1)	5
TRUNCATE(n,d)	Truncate to d decimals (no round)	TRUNCATE(1.999,2)	1.99
ABS(n)	Absolute value	ABS(-42)	42
MOD(a,b)	Remainder (like % operator)	MOD(10,3)	1
POWER(b,e)	b raised to power e	POWER(2,10)	1024
POW(b,e)	Alias for POWER	POW(3,3)	27
SQRT(n)	Square root	SQRT(144)	12
EXP(n)	e raised to power n	EXP(1)	2.71828
LOG(n)	Natural logarithm (base e)	LOG(2.71828)	1
LOG2(n)	Base-2 logarithm	LOG2(8)	3
LOG10(n)	Base-10 logarithm	LOG10(1000)	3
SIGN(n)	Sign: -1, 0, or 1	SIGN(-42)	-1
GREATEST(a,b,...)	Largest of list	GREATEST(5,2,9,1)	9
LEAST(a,b,...)	Smallest of list	LEAST(5,2,9,1)	1
PI()	Value of π	PI()	3.141593
RAND()	Random float 0 ≤ x < 1	RAND()	0.721...
RAND(seed)	Seeded random (reproducible)	RAND(42)	0.651...
FORMAT(n,d)	Format with commas	FORMAT(1234567,2)	1,234,567.00
CONV(n,from,to)	Convert number base	CONV('FF',16,10)	255
BIN(n)	Binary string of integer	BIN(255)	11111111
OCT(n)	Octal string of integer	OCT(8)	10
HEX(n)	Hexadecimal of integer	HEX(255)	FF

Example 1: ROUND, FLOOR, CEILING, TRUNCATE

▶ courses table — price column

title	price
Python	1499.00
ML A-Z	2999.00
UI/UX	999.00

```
SELECT
  title,
  price,
  ROUND(price, 0)        AS rounded,         -- 1499
  ROUND(price, -2)       AS rounded_100s,    -- 1500
  FLOOR(price)           AS floored,         -- 1499
  CEILING(price)         AS ceiled,          -- 1499 (already integer)
  TRUNCATE(price, 0)     AS truncated,       -- 1499 (no rounding!)
  ROUND(price * 1.18, 2) AS with_gst         -- 1768.82
FROM courses
WHERE level = 'Beginner'
ORDER BY price;
```

Output:

```
title                     | price   | rounded | rounded_100s | floored | ceiled | truncated |
with_gst
--------------------------+---------+---------+--------------+---------+--------+-----------+----
-----
Excel for Business        | 699.00  | 699     | 700          | 699     | 699    | 699       |
824.82
UI/UX Design Fund.        | 999.00  | 999     | 1000         | 999     | 999    | 999       |
1178.82
Python for Data Science   | 1499.00 | 1499    | 1500         | 1499    | 1499   | 1499      |
1768.82
```

How it works: ROUND(-2) rounds to the nearest hundred. TRUNCATE simply chops decimal digits without rounding — TRUNCATE(1.999,0) = 1 (not 2). FLOOR always goes down; CEILING always goes up.

Example 2: RAND — Random Records

```
-- Pick 3 random courses (different every run)
SELECT title, price FROM courses ORDER BY RAND() LIMIT 3;

-- Seeded random (same result every time — for testing)
SELECT title FROM courses ORDER BY RAND(42) LIMIT 3;

-- Generate a random integer between 1 and 100:
SELECT FLOOR(1 + RAND() * 100) AS random_int;

-- Assign random rating for demo data:
SELECT full_name, FLOOR(1 + RAND() * 5) AS simulated_rating FROM students;
```

Output:

```
-- Random 3 courses (example):
Machine Learning A-Z    | 2999.00
Excel for Business      | 699.00
React JS Complete Guide | 1999.00

-- Seeded RAND(42) always returns same 3 courses

-- random_int (example): 73
```

```
-- simulated_rating: varies per row
```

Example 3: ABS, SIGN, GREATEST, LEAST

```
SELECT
  -- ABS: useful for calculating differences without caring about sign
  ABS(price - 1500)                    AS distance_from_1500,
  -- SIGN: tells direction
  SIGN(price - 1500)                   AS above_1500,   -- 1=above, 0=equal, -1=below
  -- GREATEST/LEAST: across columns
  GREATEST(price, 1000)                AS at_least_1000,-- prices can't go below 1000
  LEAST(price, 2000)                   AS capped_2000   -- prices can't exceed 2000
FROM courses ORDER BY price;
```

Output:

```
price   | distance | above_1500 | at_least_1000 | capped_2000
--------+----------+------------+---------------+------------
699.00  | 801.00   | -1         | 1000.00       | 699.00
999.00  | 501.00   | -1         | 1000.00       | 999.00
1299.00 | 201.00   | -1         | 1299.00       | 1299.00
1499.00 | 1.00     | -1         | 1499.00       | 1499.00
1799.00 | 299.00   | 1          | 1799.00       | 1799.00
1999.00 | 499.00   | 1          | 1999.00       | 1999.00
2299.00 | 799.00   | 1          | 2299.00       | 2000.00
2499.00 | 999.00   | 1          | 2499.00       | 2000.00
```

3. Date & Time Functions — Complete Reference

Interview-Ready Definition

Date and time functions extract, format, calculate, and compare temporal data. Essential for reporting (monthly sales, yearly growth), business logic (subscription expiry, SLAs), and data analysis.

Date/Time Functions Reference Table

Function	Purpose	Example	Result (approx)
NOW()	Current date AND time	NOW()	2024-03-15 14:30:00
CURDATE()	Current date only	CURDATE()	2024-03-15
CURRENT_DATE()	Alias for CURDATE()	CURRENT_DATE()	2024-03-15
CURTIME()	Current time only	CURTIME()	14:30:00
SYSDATE()	System date+time at execution	SYSDATE()	2024-03-15 14:30:01
DATE(datetime)	Extract date part	DATE('2024-03-15 14:30:00')	2024-03-15
TIME(datetime)	Extract time part	TIME('2024-03-15 14:30:00')	14:30:00
YEAR(date)	Extract year	YEAR('2024-03-15')	2024
MONTH(date)	Extract month number	MONTH('2024-03-15')	3
DAY(date)	Extract day of month	DAY('2024-03-15')	15
HOUR(time)	Extract hour	HOUR('14:30:00')	14
MINUTE(time)	Extract minute	MINUTE('14:30:00')	30
SECOND(time)	Extract second	SECOND('14:30:45')	45
DAYNAME(date)	Name of weekday	DAYNAME('2024-03-15')	Friday
MONTHNAME(date)	Name of month	MONTHNAME('2024-03-15')	March
DAYOFWEEK(date)	Day number 1=Sun,7=Sat	DAYOFWEEK('2024-03-15')	6 (Friday)
DAYOFMONTH(date)	Day of month (same as DAY())	DAYOFMONTH('2024-03-15')	15
DAYOFYEAR(date)	Day number in year (1-366)	DAYOFYEAR('2024-03-15')	75
WEEK(date)	Week number in year	WEEK('2024-03-15')	11
WEEKOFYEAR(date)	ISO week number	WEEKOFYEAR('2024-03-15')	11
QUARTER(date)	Quarter number (1-4)	QUARTER('2024-03-15')	1
DATEDIFF(a,b)	Days between two dates	DATEDIFF('2024-12-31','2024-01-01')	365
TIMEDIFF(a,b)	Time difference as TIME	TIMEDIFF('14:00:00','09:00:00')	05:00:00
DATE_ADD(d,INTERVAL n unit)	Add interval to date	DATE_ADD('2024-01-01',INTERVAL 1 MONTH)	2024-02-01
DATE_SUB(d,INTERVAL n unit)	Subtract interval	DATE_SUB('2024-03-15',INTERVAL 7 DAY)	2024-03-08

Function	Purpose	Example	Result (approx)
ADDDATE(d,n)	Add n days to date	ADDDATE('2024-01-01',30)	2024-01-31
SUBDATE(d,n)	Subtract n days from date	SUBDATE('2024-03-15',10)	2024-03-05
DATE_FORMAT(d,fmt)	Format date as string	DATE_FORMAT(NOW(),'%d/%m/%Y')	15/03/2024
STR_TO_DATE(s,fmt)	Parse string into DATE	STR_TO_DATE('15-03-2024','%d-%m-%Y')	2024-03-15
LAST_DAY(date)	Last day of the month	LAST_DAY('2024-02-01')	2024-02-29
MAKEDATE(y,d)	Date from year + day of year	MAKEDATE(2024,75)	2024-03-15
TIMESTAMPDIFF(u,a,b)	Difference in specified unit	TIMESTAMPDIFF(MONTH,'2023-01-01','2024-03-01')	14
UNIX_TIMESTAMP()	Current Unix timestamp (seconds)	UNIX_TIMESTAMP()	1710504600
FROM_UNIXTIME(ts)	Convert Unix ts to datetime	FROM_UNIXTIME(1710504600)	2024-03-15 14:30:00
PERIOD_DIFF(p1,p2)	Months between YYYYMM periods	PERIOD_DIFF(202403,202301)	14
TO_DAYS(date)	Days since year 0	TO_DAYS('2024-03-15')	739327
FROM_DAYS(n)	Date from days since year 0	FROM_DAYS(739327)	2024-03-15
EXTRACT(unit FROM d)	Extract part using SQL standard	EXTRACT(YEAR FROM '2024-03-15')	2024

DATE_FORMAT Format Codes

Code	Meaning	Example
%Y	4-digit year	2024
%y	2-digit year	24
%m	Month 01-12	03
%M	Full month name	March
%b	Short month name	Mar
%d	Day 01-31	15
%D	Day with suffix	15th
%W	Full weekday name	Friday
%a	Short weekday	Fri
%H	Hour 00-23	14
%h	Hour 01-12	02
%i	Minutes 00-59	30
%s	Seconds 00-59	45
%p	AM or PM	PM

Code	Meaning	Example
%T	HH:MM:SS	14:30:45
%r	hh:mm:ss AM/PM	02:30:45 PM
%j	Day of year 001-366	075
%U	Week 00-53 (Sun first)	11
%V	Week 01-53 (Mon first, ISO)	11

Example 1: NOW, CURDATE, YEAR, MONTH, DAY, DATEDIFF

▶ students table — joined_date column

student_id	full_name	joined_date
1	Amit Joshi	2022-01-10
9	Rohit Sharma	2020-12-01
4	Anjali Singh	2023-02-05

```
SELECT
  full_name,
  joined_date,
  YEAR(joined_date)                              AS join_year,
  MONTHNAME(joined_date)                         AS join_month,
  DAY(joined_date)                               AS join_day,
  DAYNAME(joined_date)                           AS join_weekday,
  DATEDIFF(CURDATE(), joined_date)               AS days_on_platform,
  TIMESTAMPDIFF(MONTH, joined_date, CURDATE())   AS months_member,
  TIMESTAMPDIFF(YEAR,  joined_date, CURDATE())   AS years_member
FROM students
ORDER BY joined_date;
```

Output:

```
full_name     | joined_date | join_year | join_month | join_day | join_weekday | days | months |
years
--------------+-------------+-----------+------------+----------+--------------+------+--------+-
-----
Rohit Sharma  | 2020-12-01  | 2020      | December   | 1        | Tuesday      | 1201 | 39     |
3
Rahul Gupta   | 2021-11-20  | 2021      | November   | 20       | Saturday     | 848  | 27     |
2
Deepak Verma  | 2021-08-25  | 2021      | August     | 25       | Wednesday    | 934  | 30     |
2
Amit Joshi    | 2022-01-10  | 2022      | January    | 10       | Monday       | 795  | 26     |
2
...
```

How it works: DATEDIFF returns integer days. TIMESTAMPDIFF(MONTH,...) returns complete months. TIMESTAMPDIFF(YEAR,...) returns complete years — not just dividing days by 365.

Example 2: DATE_FORMAT — Custom Display Formats

▶ orders table — order_date column

order_id	order_date
1	2022-02-10 10:00:00

order_id	order_date
3	2022-05-01 09:00:00
15	2023-06-01 12:00:00

```
SELECT
  order_id,
  order_date,
  DATE_FORMAT(order_date, '%d %M %Y')           AS display_date,
  DATE_FORMAT(order_date, '%W, %d %b %Y')       AS long_format,
  DATE_FORMAT(order_date, '%d/%m/%Y')           AS indian_format,
  DATE_FORMAT(order_date, '%H:%i %p')           AS time_12hr,
  DATE_FORMAT(order_date, '%Y-%m')              AS year_month
FROM orders
ORDER BY order_date
LIMIT 4;
```

Output:

```
order_id | display_date    | long_format                | indian_format | time_12hr | year_month
---------+-----------------+----------------------------+---------------+-----------+-----------
14       | 15 January 2021 | Friday, 15 Jan 2021        | 15/01/2021    | 09:00 AM  | 2021-01
12       | 10 September 2021| Saturday, 10 Sep 2021     | 10/09/2021    | 10:00 AM  | 2021-09
5        | 20 December 2021| Monday, 20 Dec 2021        | 20/12/2021    | 14:00 PM  | 2021-12
6        | 05 January 2022 | Wednesday, 05 Jan 2022     | 05/01/2022    | 08:00 AM  | 2022-01
```

Example 3: DATE_ADD, DATE_SUB, LAST_DAY — Business Logic

```
SELECT
  order_id,
  order_date,
  -- 30-day refund deadline
  DATE_ADD(order_date, INTERVAL 30 DAY)      AS refund_deadline,
  -- 1 year subscription expiry
  DATE_ADD(order_date, INTERVAL 1 YEAR)      AS annual_expiry,
  -- Last day of the order month
  LAST_DAY(order_date)                       AS month_end,
  -- First day of next month
  DATE_ADD(LAST_DAY(order_date), INTERVAL 1 DAY) AS next_month_start,
  -- Days since order
  DATEDIFF(CURDATE(), order_date)            AS days_since_order,
  -- Orders placed in last 365 days?
  IF(order_date >= DATE_SUB(CURDATE(), INTERVAL 365 DAY), 'Recent','Old') AS recency
FROM orders
ORDER BY order_date DESC LIMIT 4;
```

Output:

```
order_id | order_date          | refund_deadline     | annual_expiry       | month_end  |
next_month_start | days | recency
---------+---------------------+---------------------+---------------------+------------+--------
----------+------+--------
15       | 2023-06-01 12:00:00 | 2023-07-01 12:00:00 | 2024-06-01 12:00:00 | 2023-06-30 | 2023-
07-01       | 287  | Old
11       | 2023-02-20 13:00:00 | 2023-03-22 13:00:00 | 2024-02-20 13:00:00 | 2023-02-28 | 2023-
03-01       | 388  | Old
10       | 2023-01-15 11:00:00 | 2023-02-14 11:00:00 | 2024-01-15 11:00:00 | 2023-01-31 | 2023-
02-01       | 424  | Old
```

How it works: DATE_ADD accepts INTERVAL units: SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR. LAST_DAY() handles month-end correctly including February in leap years.

Example 4: Monthly Revenue Report — Real-World Analytics

```
SELECT
  YEAR(order_date)                                AS yr,
  MONTH(order_date)                               AS mn,
  MONTHNAME(order_date)                           AS month_name,
  QUARTER(order_date)                             AS quarter,
  COUNT(*)                                        AS total_orders,
  SUM(amount_paid)                                AS total_revenue,
  ROUND(AVG(amount_paid), 2)                      AS avg_order_value
FROM orders
WHERE status = 'completed'
GROUP BY YEAR(order_date), MONTH(order_date), MONTHNAME(order_date), QUARTER(order_date)
ORDER BY yr, mn;
```

Output:

yr	mn	month_name	quarter	total_orders	total_revenue	avg_order_value
2021	1	January	1	1	3499.00	3499.00
2021	9	September	3	1	1999.00	1999.00
2021	12	December	4	1	1499.00	1499.00
2022	2	February	1	1	1499.00	1499.00
2022	4	April	2	1	1799.00	1799.00
2022	5	May	2	1	2999.00	2999.00
2022	8	August	3	1	1799.00	1799.00
2022	9	September	3	1	699.00	699.00
2022	10	October	4	0	0	0
2023	1	January	1	1	1499.00	1499.00
2023	2	February	1	2	4298.00	2149.00
2023	3	March	1	1	2499.00	2499.00
2023	6	June	2	1	2299.00	2299.00

? What is the difference between NOW() and CURDATE()?

✅NOW() returns a DATETIME with both date and time (2024-03-15 14:30:00). CURDATE() returns only the DATE portion (2024-03-15). Use CURDATE() for date-level comparisons to avoid time mismatches: WHERE joined_date = CURDATE() vs WHERE joined_date = NOW() (the latter rarely matches exactly).

? What is the difference between DATEDIFF() and TIMESTAMPDIFF()?

✅DATEDIFF(a,b) always returns the difference in DAYS (integer). TIMESTAMPDIFF(unit, a, b) returns the difference in any specified unit: SECOND, MINUTE, HOUR, DAY, MONTH, YEAR. TIMESTAMPDIFF counts complete units — TIMESTAMPDIFF(YEAR, '2023-01-01', '2024-01-01') = 1, but TIMESTAMPDIFF(YEAR, '2023-06-01', '2024-01-01') = 0 (not a full year).

4. Subqueries — All Types

Interview-Ready Definition

A subquery is a SELECT statement nested inside another SQL statement. Types: Scalar (returns one value), Multi-row (returns multiple values for IN/ANY/ALL), Correlated (references outer query, runs per row), Derived Table (subquery in FROM clause).

Subquery Types Summary

Type	Returns	Used In	Runs Once?
Scalar	Single value (1 row × 1 col)	SELECT, WHERE, HAVING	Yes
Multi-row	Multiple values (many rows × 1 col)	IN, ANY, ALL	Yes
Multi-column	Multiple columns	Rarely (special cases)	Yes
Correlated	Value depending on outer row	WHERE, HAVING	No — once per outer row
Derived Table	A virtual table	FROM clause	Yes (once)

Example 1: Scalar Subquery — Single Value in WHERE

▶ courses table — all 10 rows

title	price	level
Python	1499	Beginner
ML A-Z	2999	Advanced
React JS	1999	Intermediate
Deep Learning	3499	Advanced

```
-- Find all courses priced above the AVERAGE price
SELECT title, price, level
FROM courses
WHERE price > (SELECT AVG(price) FROM courses)   -- scalar: returns one number
ORDER BY price DESC;

-- The subquery: SELECT AVG(price) FROM courses → 1959.00 (10 courses average)
```

Output:

```
Subquery first: AVG(price) = 1959.00
Then outer WHERE: price > 1959.00

title                    | price    | level
-------------------------+----------+----------
Deep Learning            | 3499.00 | Advanced
Machine Learning A-Z     | 2999.00 | Advanced
Flutter Mobile Dev       | 2499.00 | Advanced
Advanced React Patterns  | 2299.00 | Advanced
React JS Complete Guide  | 1999.00 | Intermediate
```

How it works: The scalar subquery AVG(price) runs ONCE and returns one number. The outer query uses that number as a constant in its WHERE condition.

Example 2: Multi-Row Subquery — IN Operator

```
-- Find students who bought an Advanced-level course
SELECT DISTINCT s.full_name, s.email
FROM students s
WHERE s.student_id IN (
    SELECT o.student_id          -- returns multiple student_ids
    FROM orders o
    JOIN courses c ON o.course_id = c.course_id
    WHERE c.level = 'Advanced'
      AND o.status = 'completed'
)
ORDER BY s.full_name;
```

Output:

```
Subquery returns: (4, 6, 9, 10)  [student_ids who bought Advanced courses]

full_name     | email
--------------+-------------------
Anjali Singh  | anjali@gmail.com
Kavya Reddy   | kavya@gmail.com
Pooja Iyer    | pooja@gmail.com
Rohit Sharma  | rohit@gmail.com
```

How it works: The inner SELECT returns multiple rows (student_ids). The outer WHERE IN filters to only those students. Equivalent to a JOIN but sometimes more readable for filtering logic.

Example 3: Correlated Subquery — Runs Once Per Row

```
-- Find courses priced ABOVE their own category average
SELECT
  c.title,
  c.price,
  c.level,
  cat.category_name,
  ROUND((SELECT AVG(c2.price) FROM courses c2
         WHERE c2.category_id = c.category_id), 2) AS category_avg
FROM courses c
JOIN categories cat ON c.category_id = cat.category_id
WHERE c.price > (
    SELECT AVG(c2.price)
    FROM courses c2
    WHERE c2.category_id = c.category_id   -- ← references outer c.category_id!
)
ORDER BY cat.category_name, c.price DESC;
```

Output:

```
title                   | price   | level    | category_name    | category_avg
------------------------+---------+----------+------------------+-------------
Deep Learning           | 3499.00 | Advanced | Data Science     | 2665.67
Machine Learning A-Z    | 2999.00 | Advanced | Data Science     | 2665.67
Advanced React Patterns | 2299.00 | Advanced | Web Development  | 1865.67
Business Analytics      | 1799.00 | Intermed | Business         | 1249.00
Flutter Mobile Dev      | 2499.00 | Advanced | Mobile Dev       | 2499.00
```

How it works: The correlated subquery references c.category_id from the OUTER query. It runs ONCE PER ROW of the outer query — for each course it calculates that course's category average. Python (1499) is below Data Science average (2665.67) so it's excluded.

⚠ **Pitfall:** Correlated subqueries can be slow on large tables — they run N times (once per outer row). Consider using window functions (AVG OVER PARTITION BY) as a faster alternative.

Example 4: Derived Table — Subquery in FROM Clause

```
-- Find students whose total spending is above the average student spending
SELECT
  s.full_name,
  student_totals.total_spent,
  student_totals.order_count
FROM students s
JOIN (
    -- This subquery becomes a virtual table 'student_totals'
    SELECT
        student_id,
        SUM(amount_paid)  AS total_spent,
        COUNT(*)          AS order_count
    FROM orders
    WHERE status = 'completed'
    GROUP BY student_id
) AS student_totals ON s.student_id = student_totals.student_id
WHERE student_totals.total_spent > (
    SELECT AVG(total_per_student)
    FROM (
        SELECT student_id, SUM(amount_paid) AS total_per_student
        FROM orders WHERE status = 'completed'
        GROUP BY student_id
    ) AS avg_calc
)
ORDER BY student_totals.total_spent DESC;
```

Output:

```
full_name     | total_spent | order_count
-------------+------------+------------
Kavya Reddy   | 4498.00     | 2
Meera Nair    | 4298.00     | 2
Rohit Sharma  | 3499.00     | 1
Amit Joshi    | 3298.00     | 2
```

How it works: A subquery in FROM must be given an alias (student_totals). It runs once and acts like a table. This is called a "derived table." Note: In modern MySQL, CTEs (WITH clause) are cleaner for this pattern.

Example 5: Scalar Subquery in SELECT — Per-Row Lookup

```
-- Show each course with its category name and instructor name
-- Using scalar subqueries instead of JOINs
SELECT
  course_id,
  title,
  price,
  (SELECT category_name FROM categories WHERE category_id = c.category_id) AS category,
  (SELECT full_name     FROM instructors WHERE instructor_id = c.instructor_id) AS
instructor
FROM courses c
ORDER BY price DESC LIMIT 4;
```

Output:

```
course_id | title                    | price    | category      | instructor
----------+-------------------------+---------+--------------+-------------
10        | Deep Learning            | 3499.00 | Data Science | Arjun Mehta
2         | Machine Learning A-Z     | 2999.00 | Data Science | Arjun Mehta
6         | Flutter Mobile Dev       | 2499.00 | Mobile Dev   | Arjun Mehta
8         | Advanced React Patterns | 2299.00 | Web Dev      | Priya Sharma
```

How it works: Scalar subqueries in SELECT run once per row. Result: same as JOIN but less efficient for large datasets. Use JOINs in production; scalar subqueries in SELECT are useful for quick lookups.

? When should you use a subquery vs a JOIN?

✅Use a JOIN when you need columns from both tables in the result — more efficient. Use a subquery when you need to filter by a derived value (average, max) that doesn't need to appear in output. Correlated subqueries are powerful but slow — replace with window functions or CTEs where possible.

5. Window Functions — Complete Guide

Interview-Ready Definition

Window functions perform calculations across a set of rows related to the current row WITHOUT collapsing rows (unlike GROUP BY). Syntax: function() OVER (PARTITION BY ... ORDER BY ...). They are the most powerful analytical tool in SQL.

Window Function Syntax

```
function_name()  OVER (
    [PARTITION BY col1, col2]     -- divide into groups (like GROUP BY but keeps all rows)
    [ORDER BY col3 ASC|DESC]      -- defines row order within each partition
    [ROWS|RANGE BETWEEN frame_start AND frame_end]  -- sliding window frame
)
```

Ranking Functions Comparison

Function	Ties Behavior	Example scores: 90,90,80,70	Gap after tie?
ROW_NUMBER()	No ties — unique sequence	1, 2, 3, 4	N/A
RANK()	Tied rows get same rank	1, 1, 3, 4	Yes — skips 2
DENSE_RANK()	Tied rows get same rank	1, 1, 2, 3	No — no gap
NTILE(n)	Divides into n equal buckets	1,1,2,2 (NTILE(2))	N/A

Example 1: ROW_NUMBER, RANK, DENSE_RANK — Side-by-Side

▶ **courses table — price column**

title	price
Deep Learning	3499
ML A-Z	2999
Flutter	2499
React Adv	2299
React JS	1999

```
SELECT
  title,
  price,
  ROW_NUMBER()  OVER (ORDER BY price DESC) AS row_num,
  RANK()        OVER (ORDER BY price DESC) AS rnk,
  DENSE_RANK()  OVER (ORDER BY price DESC) AS dense_rnk
FROM courses
ORDER BY price DESC;
```

Output:

```
title                     | price    | row_num | rnk | dense_rnk
--------------------------+---------+---------+-----+----------
Deep Learning             | 3499.00 | 1       | 1   | 1
```

```
Machine Learning A-Z       | 2999.00 | 2        | 2    | 2
Flutter Mobile Dev         | 2499.00 | 3        | 3    | 3
Advanced React Patterns    | 2299.00 | 4        | 4    | 4
React JS Complete Guide    | 1999.00 | 5        | 5    | 5
Business Analytics         | 1799.00 | 6        | 6    | 6
Python for Data Science    | 1499.00 | 7        | 7    | 7
MySQL Mastery              | 1299.00 | 8        | 8    | 8
UI/UX Design Fund.         | 999.00  | 9        | 9    | 9
Excel for Business         | 699.00  | 10       | 10   | 10

(All prices unique here — to see tie differences, add same price for two courses:)
If two courses have price 2999:
ROW_NUMBER: 1,2,3,4...  (no ties)
RANK:        1,2,2,4...  (skip 3 after tie)
DENSE_RANK:  1,2,2,3...  (no skip)
```

Example 2: PARTITION BY — Rank Within Each Category

▶ courses with category_id

course_id	title	category_id	price
1	Python	1	1499
2	ML A-Z	1	2999
10	Deep Learning	1	3499
3	React JS	2	1999
5	MySQL	2	1299
8	React Adv	2	2299

```sql
SELECT
  cat.category_name,
  c.title,
  c.price,
  RANK()       OVER (PARTITION BY c.category_id ORDER BY c.price DESC) AS rank_in_cat,
  DENSE_RANK() OVER (PARTITION BY c.category_id ORDER BY c.price DESC) AS dense_rank_cat,
  ROW_NUMBER() OVER (PARTITION BY c.category_id ORDER BY c.price DESC) AS row_in_cat
FROM courses c
JOIN categories cat ON c.category_id = cat.category_id
ORDER BY cat.category_name, c.price DESC;
```

Output:

```
category_name    | title                    | price    | rank_in_cat | dense_rank_cat | row_in_cat
-----------------+--------------------------+----------+-------------+----------------+-----------
Data Science     | Deep Learning            | 3499.00 | 1           | 1              | 1
Data Science     | Machine Learning A-Z     | 2999.00 | 2           | 2              | 2
Data Science     | Python for Data Science | 1499.00 | 3           | 3              | 3
Web Development  | Advanced React Patterns | 2299.00 | 1           | 1              | 1
Web Development  | React JS Complete Guide | 1999.00 | 2           | 2              | 2
Web Development  | MySQL Mastery            | 1299.00 | 3           | 3              | 3
Business         | Business Analytics       | 1799.00 | 1           | 1              | 1
Business         | Excel for Business       | 699.00  | 2           | 2              | 2
Design           | UI/UX Design Fund.       | 999.00  | 1           | 1              | 1
Mobile Dev       | Flutter Mobile Dev       | 2499.00 | 1           | 1              | 1
```

How it works: PARTITION BY category_id restarts the rank counter for each category. Deep Learning is rank 1 in Data Science; Advanced React is rank 1 in Web Dev. All original rows are preserved — unlike GROUP BY.

Example 3: SUM OVER — Running Total and Cumulative Revenue

▶ orders by date (subset)

order_id	order_date	amount_paid
14	2021-01-15	3499
5	2021-12-20	1499
12	2021-09-10	1999
1	2022-02-10	1499

```
SELECT
  order_id,
  DATE(order_date)                          AS order_day,
  amount_paid,
  -- Running total (cumulative sum ordered by date)
  SUM(amount_paid) OVER (
      ORDER BY order_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  )                                         AS running_total,
  -- Running average
  ROUND(AVG(amount_paid) OVER (
      ORDER BY order_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ), 2)                                     AS running_avg,
  -- Cumulative count
  COUNT(*) OVER (ORDER BY order_date)       AS order_count_so_far
FROM orders
WHERE status = 'completed'
ORDER BY order_date;
```

Output:

```
order_id | order_day  | amount_paid | running_total | running_avg | order_count_so_far
---------+------------+-------------+---------------+-------------+-------------------
14       | 2021-01-15 | 3499.00     | 3499.00       | 3499.00     | 1
12       | 2021-09-10 | 1999.00     | 5498.00       | 2749.00     | 2
5        | 2021-12-20 | 1499.00     | 6997.00       | 2332.33     | 3
6        | 2022-01-05 | 999.00      | 7996.00       | 1999.00     | 4    (refunded but...)
1        | 2022-02-10 | 1499.00     | 9495.00       | 1899.00     | 5
...
```

How it works: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW = "from the first row up to and including the current row" — this is the standard running total frame. As each row is processed, the sum grows.

Example 4: LAG and LEAD — Compare with Previous/Next Row

▶ orders by student and date

student_id	order_date	amount_paid
1	2022-02-10	1499
1	2022-04-05	1799
2	2022-05-01	2999
2	2023-02-14	1299

```
SELECT
  s.full_name,
```

```
  o.order_date,
  o.amount_paid,
  -- Previous order amount for this student
  LAG(o.amount_paid)  OVER (PARTITION BY o.student_id ORDER BY o.order_date)
                                                AS prev_order_amt,
  -- Next order amount for this student
  LEAD(o.amount_paid) OVER (PARTITION BY o.student_id ORDER BY o.order_date)
                                                AS next_order_amt,
  -- Change from previous order
  o.amount_paid - LAG(o.amount_paid) OVER (PARTITION BY o.student_id ORDER BY
o.order_date)
                                                AS change_from_prev
FROM orders o
JOIN students s ON o.student_id = s.student_id
ORDER BY s.full_name, o.order_date;
```

Output:

```
full_name   | order_date          | amount_paid | prev_order_amt | next_order_amt |
change_from_prev
------------+---------------------+-------------+----------------+----------------+-------------
--
Amit Joshi  | 2022-02-10 10:00:00 | 1499.00     | NULL           | 1799.00        | NULL
Amit Joshi  | 2022-04-05 11:30:00 | 1799.00     | 1499.00        | NULL           | 300.00
Kavya Reddy | 2023-01-15 11:00:00 | 1499.00     | NULL           | 2999.00        | NULL
Kavya Reddy | 2023-02-20 13:00:00 | 2999.00     | 1499.00        | NULL           | 1500.00
Meera Nair  | 2022-05-01 09:00:00 | 2999.00     | NULL           | 1299.00        | NULL
Meera Nair  | 2023-02-14 15:00:00 | 1299.00     | 2999.00        | NULL           | -1700.00
```

How it works: LAG accesses the previous row value within the partition. LEAD accesses the next row. First/last rows show NULL (no previous/next). change_from_prev shows spending trends per student. LAG/LEAD accept optional offset and default: LAG(col, 2, 0).

Example 5: FIRST_VALUE, LAST_VALUE, NTH_VALUE

```
SELECT
  cat.category_name,
  c.title,
  c.price,
  -- Most expensive course in this category
  FIRST_VALUE(c.title) OVER (
      PARTITION BY c.category_id
      ORDER BY c.price DESC
  )                                          AS most_expensive_in_cat,
  -- Cheapest course in this category
  LAST_VALUE(c.title) OVER (
      PARTITION BY c.category_id
      ORDER BY c.price DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  )                                          AS cheapest_in_cat,
  -- 2nd most expensive in this category
  NTH_VALUE(c.title, 2) OVER (
      PARTITION BY c.category_id
      ORDER BY c.price DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  )                                          AS second_in_cat
FROM courses c
JOIN categories cat ON c.category_id = cat.category_id
ORDER BY cat.category_name, c.price DESC;
```

Output:

```
category_name   | title                   | price   | most_expensive_in_cat   | cheapest_in_cat
| second_in_cat
----------------+-------------------------+---------+-------------------------+-----------------
-------+-------------
Data Science    | Deep Learning           | 3499.00 | Deep Learning           | Python for Data
Science | ML A-Z
```

```
Data Science     | Machine Learning A-Z     | 2999.00 | Deep Learning            | Python for Data
Science | ML A-Z
Data Science     | Python for Data Science | 1499.00 | Deep Learning            | Python for Data
Science | ML A-Z
Web Development | Advanced React Patterns | 2299.00 | Advanced React Patterns | MySQL Mastery
| React JS
```

How it works: FIRST_VALUE is straightforward. LAST_VALUE requires ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING to look at the entire partition — without this frame it only sees up to the current row. NTH_VALUE(col, 2) gets the second row in the ordered partition.

Example 6: NTILE — Divide into Quartiles/Percentiles

```
SELECT
  title,
  price,
  NTILE(4) OVER (ORDER BY price DESC) AS price_quartile,
  NTILE(3) OVER (ORDER BY price DESC) AS price_tercile,
  CASE NTILE(4) OVER (ORDER BY price DESC)
    WHEN 1 THEN 'Top 25% (Elite)'
    WHEN 2 THEN 'Upper-Mid 25%'
    WHEN 3 THEN 'Lower-Mid 25%'
    WHEN 4 THEN 'Bottom 25% (Budget)'
  END AS price_segment
FROM courses
ORDER BY price DESC;
```

Output:

```
title                      | price    | price_quartile | price_tercile | price_segment
---------------------------+----------+----------------+---------------+-----------------------
Deep Learning              | 3499.00 | 1               | 1             | Top 25% (Elite)
Machine Learning A-Z       | 2999.00 | 1               | 1             | Top 25% (Elite)
Flutter Mobile Dev         | 2499.00 | 1               | 1             | Top 25% (Elite)
Advanced React Patterns    | 2299.00 | 2               | 2             | Upper-Mid 25%
React JS Complete Guide    | 1999.00 | 2               | 2             | Upper-Mid 25%
Business Analytics         | 1799.00 | 2               | 2             | Upper-Mid 25%
Python for Data Science    | 1499.00 | 3               | 3             | Lower-Mid 25%
MySQL Mastery              | 1299.00 | 3               | 3             | Lower-Mid 25%
UI/UX Design Fund.         | 999.00  | 4               | 3             | Bottom 25% (Budget)
Excel for Business         | 699.00  | 4               | 3             | Bottom 25% (Budget)
```

? What is the difference between GROUP BY and PARTITION BY?

✅GROUP BY COLLAPSES rows into one row per group — you lose individual row details. PARTITION BY (in window functions) DIVIDES rows into groups but KEEPS all rows — each row still appears individually but gets a calculated value based on its group. Use GROUP BY for summarisation; PARTITION BY for per-row analytics alongside row details.

? What is the difference between RANK() and DENSE_RANK()?

✅When tied rows exist: RANK() assigns the same rank to ties but SKIPS the next rank(s) — scores 90,90,80 get ranks 1,1,3. DENSE_RANK() assigns the same rank to ties but does NOT skip — same scores get 1,1,2. Use DENSE_RANK() when you want consecutive positions without gaps.

6. Common Table Expressions (CTEs)

Interview-Ready Definition

A CTE (Common Table Expression) is a temporary named result set defined with the WITH keyword. It exists only for the duration of the query. CTEs make complex queries readable by breaking them into named, logical steps. Recursive CTEs can traverse hierarchical data.

Example 1: Basic CTE — Courses Above Average Price

```
-- WITHOUT CTE (hard to read — subquery inline):
SELECT title, price FROM courses
WHERE price > (SELECT AVG(price) FROM courses);

-- WITH CTE (clear, readable):
WITH avg_price AS (
    SELECT AVG(price) AS avg_val FROM courses
)
SELECT c.title, c.price
FROM courses c, avg_price
WHERE c.price > avg_price.avg_val
ORDER BY c.price DESC;
```

Output:

```
title                    | price
-------------------------+---------
Deep Learning            | 3499.00
Machine Learning A-Z     | 2999.00
Flutter Mobile Dev       | 2499.00
Advanced React Patterns  | 2299.00
React JS Complete Guide  | 1999.00
```

How it works: The CTE avg_price is defined once with WITH, then used in the main query like a table. It makes the intent clear: first compute the average, then filter by it.

Example 2: Multiple CTEs — Pipeline of Steps

```
WITH
-- Step 1: Student order totals
student_totals AS (
    SELECT
        student_id,
        COUNT(*)           AS order_count,
        SUM(amount_paid)   AS total_spent
    FROM orders
    WHERE status = 'completed'
    GROUP BY student_id
),
-- Step 2: Students who spent > 3000 AND have 2+ orders
high_value AS (
    SELECT student_id, total_spent, order_count
    FROM student_totals
    WHERE total_spent > 3000 AND order_count >= 2
),
-- Step 3: Get review counts for these students
review_activity AS (
    SELECT student_id, COUNT(*) AS review_count
    FROM reviews
    GROUP BY student_id
)
```

```
-- Main query: join everything together
SELECT
    s.full_name,
    hv.total_spent,
    hv.order_count,
    IFNULL(ra.review_count, 0) AS reviews_given
FROM students s
JOIN high_value hv      ON s.student_id = hv.student_id
LEFT JOIN review_activity ra ON s.student_id = ra.student_id
ORDER BY hv.total_spent DESC;
```

Output:

```
full_name     | total_spent | order_count | reviews_given
-------------+------------+-------------+-------------
Kavya Reddy   | 4498.00     | 2           | 2
Meera Nair    | 4298.00     | 2           | 2
Amit Joshi    | 3298.00     | 2           | 2
```

How it works: Multiple CTEs are separated by commas after WITH. Each CTE can reference previous CTEs. The main query at the end joins them. This reads like a pipeline: compute totals → filter high-value → join reviews. Far cleaner than nested subqueries.

Example 3: Recursive CTE — Generate a Number Series

```
-- Generate numbers 1 to 10 using recursive CTE
WITH RECURSIVE numbers AS (
    -- Anchor: start value
    SELECT 1 AS n

    UNION ALL

    -- Recursive: increment by 1, stop at 10
    SELECT n + 1
    FROM numbers
    WHERE n < 10
)
SELECT n FROM numbers;

-- Generate a date series (last 7 days):
WITH RECURSIVE date_series AS (
    SELECT DATE_SUB(CURDATE(), INTERVAL 6 DAY) AS dt
    UNION ALL
    SELECT DATE_ADD(dt, INTERVAL 1 DAY)
    FROM date_series
    WHERE dt < CURDATE()
)
SELECT dt FROM date_series;
```

Output:

```
Numbers 1-10:
n: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Last 7 dates:
dt: 2024-03-09, 2024-03-10, 2024-03-11, 2024-03-12,
    2024-03-13, 2024-03-14, 2024-03-15
```

How it works: Recursive CTEs have two parts joined by UNION ALL: the ANCHOR member (starting value, runs once) and the RECURSIVE member (references the CTE itself, repeats until WHERE condition fails). Use for number series, date ranges, org chart traversal.

Example 4: Recursive CTE — Category Hierarchy

```
-- Add parent_category_id to categories for hierarchy demo
ALTER TABLE categories ADD COLUMN parent_category_id INT DEFAULT NULL;
UPDATE categories SET parent_category_id = 1 WHERE category_id IN (10,11);
-- Assume category 10 = 'Machine Learning' is child of 1 = 'Data Science'

WITH RECURSIVE category_tree AS (
    -- Anchor: top-level categories (no parent)
    SELECT category_id, category_name, parent_category_id,
           0 AS depth,
           category_name AS path
    FROM categories
    WHERE parent_category_id IS NULL

    UNION ALL

    -- Recursive: find children
    SELECT c.category_id, c.category_name, c.parent_category_id,
           ct.depth + 1,
           CONCAT(ct.path, ' > ', c.category_name)
    FROM categories c
    JOIN category_tree ct ON c.parent_category_id = ct.category_id
)
SELECT category_id, REPEAT('  ', depth), category_name, path
FROM category_tree
ORDER BY path;
```

Output:

```
category_id | indent | category_name      | path
------------+--------+--------------------+-----------------------------
1           |        | Data Science       | Data Science
10          |   (2)  | Machine Learning   | Data Science > Machine Learning
2           |        | Web Development    | Web Development
...
```

? What is the difference between a CTE and a subquery?

✅A subquery is inline and can only be used once in the query. A CTE is NAMED and can be REFERENCED multiple times in the same query. CTEs also support recursion (WITH RECURSIVE). CTEs make complex queries more readable by separating logical steps. Performance is generally similar, but CTEs are better for code maintainability.

? What is the difference between a CTE and a temporary table?

✅A CTE exists only for a SINGLE QUERY execution — it is discarded immediately after. A TEMPORARY TABLE persists for the entire SESSION, can be indexed, and can be reused across multiple queries. CTEs are cleaner for single-query logic; temp tables are better for multi-step processing in stored procedures.

7. Views

Interview-Ready Definition

A view is a virtual table defined by a stored SELECT query. It does NOT store data — it executes its query every time it is accessed. Views simplify complex queries, implement security (restrict visible columns/rows), and provide a stable interface even if underlying tables change.

Example 1: CREATE VIEW — Simplify Complex JOIN

```
-- Create a view that hides the JOIN complexity
CREATE VIEW vw_order_details AS
SELECT
    o.order_id,
    s.full_name      AS student_name,
    s.email          AS student_email,
    c.title          AS course_title,
    cat.category_name,
    i.full_name      AS instructor_name,
    o.amount_paid,
    o.status,
    o.coupon_code,
    DATE(o.order_date) AS order_date
FROM orders o
JOIN students   s   ON o.student_id   = s.student_id
JOIN courses    c   ON o.course_id    = c.course_id
JOIN categories cat ON c.category_id  = cat.category_id
JOIN instructors i  ON c.instructor_id = i.instructor_id;

-- Now query the view as if it's a simple table:
SELECT * FROM vw_order_details WHERE status = 'completed' LIMIT 3;

-- Filter, aggregate, join views just like tables:
SELECT student_name, SUM(amount_paid) AS total
FROM vw_order_details
WHERE status = 'completed'
GROUP BY student_name
ORDER BY total DESC LIMIT 5;
```

Output:

```
student_name | course_title            | category_name | instructor_name | amount_paid | status
-------------+-------------------------+---------------+-----------------+-------------+---------
--
Amit Joshi   | Python for Data Science | Data Science  | Ravi Kumar      | 1499.00     |
completed
Amit Joshi   | React JS Complete Guide | Web Dev       | Priya Sharma    | 1799.00     |
completed
Meera Nair   | Machine Learning A-Z    | Data Science  | Arjun Mehta     | 2999.00     |
completed
```

How it works: Application code queries vw_order_details without knowing the JOIN structure. The view executes the underlying SELECT each time. If the base tables change (new columns), the view still works.

Example 2: CREATE VIEW for Security — Hide Sensitive Columns

```
-- Expose only non-sensitive student columns
CREATE VIEW vw_student_public AS
SELECT
    student_id,
    full_name,
```

```
    city,
    joined_date
    -- email and phone intentionally EXCLUDED
FROM students
WHERE is_active = 1;   -- Only active students

-- Grant access to the view, not the base table:
-- GRANT SELECT ON edushop.vw_student_public TO 'reporting_user'@'%';

-- Reporting team can see this (no email/phone exposure):
SELECT * FROM vw_student_public LIMIT 3;
```

Output:

```
student_id | full_name    | city      | joined_date
-----------+--------------+-----------+------------
1          | Amit Joshi   | Hyderabad | 2022-01-10
2          | Meera Nair   | Kochi     | 2022-03-15
3          | Rahul Gupta  | Delhi     | 2021-11-20
```

How it works: Views implement the principle of least privilege — grant access to the view but not the underlying table. Reporting users see public info; email/phone remain hidden.

Example 3: Updatable View and WITH CHECK OPTION

```
-- Simple single-table view — updatable!
CREATE VIEW vw_active_students AS
SELECT student_id, full_name, email, city, is_active
FROM students
WHERE is_active = 1
WITH CHECK OPTION;   -- prevents inserting/updating rows that would leave the view

-- This UPDATE works (updatable single-table view):
UPDATE vw_active_students SET city = 'Hyderabad' WHERE student_id = 3;

-- WITH CHECK OPTION: this UPDATE would FAIL
-- (would set is_active=0, removing row from view's WHERE condition):
UPDATE vw_active_students SET is_active = 0 WHERE student_id = 1;
-- ERROR: CHECK OPTION failed 'edushop.vw_active_students'

-- Complex views (JOINs, GROUP BY) are NOT updatable:
UPDATE vw_order_details SET amount_paid = 999 WHERE order_id = 1;
-- ERROR: The target table vw_order_details of the UPDATE is not updatable
```

How it works: Simple single-table views without GROUP BY, aggregates, DISTINCT, or JOINs are updatable. WITH CHECK OPTION prevents modifications that would make a row disappear from the view's perspective.

Example 4: DROP VIEW, ALTER VIEW, SHOW CREATE VIEW

```
-- View the view's definition
SHOW CREATE VIEW vw_order_details;

-- List all views
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- Modify a view (redefines it completely)
CREATE OR REPLACE VIEW vw_active_students AS
SELECT student_id, full_name, email, city, phone, joined_date, is_active
FROM students
WHERE is_active = 1;

-- Drop a view
```

```
DROP VIEW IF EXISTS vw_student_public;

-- Check if a view's underlying tables changed (view becomes invalid)
CHECK TABLE vw_order_details;
```

❓ What is a materialized view and does MySQL support it?

✅A materialized view stores the query RESULTS physically for fast reads — unlike a regular view which re-executes the query each time. MySQL does NOT natively support materialized views. You can simulate them with a summary table + scheduled EVENT or TRIGGER that refreshes the data periodically.

8. Triggers

⍰ Interview-Ready Definition

A trigger is a stored program that automatically executes in response to INSERT, UPDATE, or DELETE on a table. Fires BEFORE or AFTER the event, FOR EACH ROW. Uses OLD (previous values) and NEW (new values). Used for auditing, enforcing business rules, and maintaining derived data.

Trigger Reference

Trigger Type	When It Fires	OLD available?	NEW available?	Can modify NEW?
BEFORE INSERT	Before inserting new row	No	Yes	Yes — can change values before save
AFTER INSERT	After new row is saved	No	Yes	No
BEFORE UPDATE	Before row is changed	Yes	Yes	Yes — can validate/modify
AFTER UPDATE	After row is changed	Yes	Yes	No
BEFORE DELETE	Before row is removed	Yes	No	No
AFTER DELETE	After row is removed	Yes	No	No

Example 1: AFTER INSERT Trigger — Audit Log

```
-- First: create an audit table
CREATE TABLE audit_log (
    log_id      INT PRIMARY KEY AUTO_INCREMENT,
    action_type VARCHAR(20) NOT NULL,
    table_name  VARCHAR(50) NOT NULL,
    record_id   INT,
    changed_by  VARCHAR(50) DEFAULT USER(),
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    details     TEXT
);

-- Create the trigger
DELIMITER $$
CREATE TRIGGER trg_after_student_insert
AFTER INSERT ON students
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (action_type, table_name, record_id, details)
    VALUES (
        'INSERT',
        'students',
        NEW.student_id,
        CONCAT('New student registered: ', NEW.full_name, ' (', NEW.email, ') from ',
IFNULL(NEW.city,'Unknown'))
    );
END $$
DELIMITER ;

-- Test: Insert a new student
INSERT INTO students (full_name, email, city, joined_date)
```

```
VALUES ('Kiran Rao', 'kiran@gmail.com', 'Bengaluru', CURDATE());
```

Output:

```
-- students table: Kiran Rao inserted with student_id = 11


-- audit_log table automatically populated:
log_id | action_type | table_name | record_id | changed_by     | action_time         | details
-------+-------------+------------+-----------+----------------+---------------------+-----------
----------------
1      | INSERT      | students   | 11        | root@localhost | 2024-03-15 14:30:00 | New
student registered: Kiran Rao (kiran@gmail.com) from Bengaluru
```

How it works: NEW.column_name accesses the values of the row being inserted. The trigger fires automatically — no application code needed. Audit logs cannot be bypassed even if someone inserts directly via SQL.

Example 2: BEFORE UPDATE Trigger — Validate Business Rule

```
-- Prevent reducing course price by more than 50%
DELIMITER $$
CREATE TRIGGER trg_before_price_update
BEFORE UPDATE ON courses
FOR EACH ROW
BEGIN
    -- Check if the new price is less than 50% of old price
    IF NEW.price < OLD.price * 0.50 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Price cannot be reduced by more than 50% in one
update.';
    END IF;

    -- Also auto-update a last_modified timestamp if it exists
    SET NEW.published_at = NEW.published_at;  -- example of modifying NEW value
END $$
DELIMITER ;

-- Test: Try to drop price from 2999 to 100 (96.7% reduction):
UPDATE courses SET price = 100 WHERE course_id = 2;
-- ERROR 1644: ERROR: Price cannot be reduced by more than 50% in one update.

-- Test: Reduce by 30% (allowed):
UPDATE courses SET price = ROUND(2999 * 0.70, 2) WHERE course_id = 2;
-- Query OK, 1 row affected  (price = 2099.30)
```

How it works: BEFORE triggers fire before the change is saved. SIGNAL SQLSTATE '45000' raises a custom error and ABORTS the update. OLD.price is the current value, NEW.price is the attempted value. You can also MODIFY NEW values in a BEFORE trigger.

Example 3: AFTER DELETE Trigger — Cascading Soft Delete

```
-- When a student is deleted, archive their orders instead of hard delete
DELIMITER $$
CREATE TRIGGER trg_after_student_delete
AFTER DELETE ON students
FOR EACH ROW
BEGIN
    -- Log the deletion
    INSERT INTO audit_log (action_type, table_name, record_id, details)
    VALUES (
        'DELETE',
        'students',
        OLD.student_id,
        CONCAT('Student deleted: ', OLD.full_name, ' — orders archived')
```

```
    );

    -- Mark their orders as archived (soft delete)
    UPDATE orders
    SET status = 'refunded'
    WHERE student_id = OLD.student_id AND status = 'pending';
END $$
DELIMITER ;

-- View, drop, and manage triggers:
SHOW TRIGGERS FROM edushop;
SHOW TRIGGERS LIKE 'trg_%';
DROP TRIGGER IF EXISTS trg_after_student_insert;
```

❓ What is the difference between BEFORE and AFTER triggers?

✅BEFORE triggers fire BEFORE the row is saved — they can modify NEW values and can abort the operation with SIGNAL. AFTER triggers fire AFTER the row is saved — cannot modify NEW values but can perform side effects (audit log, update related tables). Use BEFORE for validation; AFTER for logging and cascading updates.

9. Stored Procedures

Interview-Ready Definition

A stored procedure is a named, pre-compiled block of SQL stored in the database. Benefits: reusability, security (grant EXECUTE without table access), reduced network traffic (logic runs on server), pre-compiled plan. Supports IN, OUT, INOUT parameters, IF/ELSE, CASE, LOOP, CURSOR.

Example 1: Simple Procedure with IN Parameter

```
DELIMITER $$
CREATE PROCEDURE GetCoursesByLevel(IN p_level VARCHAR(20))
BEGIN
    SELECT
        c.title,
        c.price,
        i.full_name  AS instructor,
        cat.category_name
    FROM courses c
    JOIN instructors i  ON c.instructor_id = i.instructor_id
    JOIN categories cat ON c.category_id   = cat.category_id
    WHERE c.level = p_level
    ORDER BY c.price DESC;
END $$
DELIMITER ;

-- Call:
CALL GetCoursesByLevel('Advanced');
```

Output:

```
title                    | price    | instructor    | category_name
-------------------------+----------+---------------+---------------
Deep Learning            | 3499.00 | Arjun Mehta   | Data Science
Machine Learning A-Z     | 2999.00 | Arjun Mehta   | Data Science
Flutter Mobile Dev       | 2499.00 | Arjun Mehta   | Mobile Dev
Advanced React Patterns | 2299.00 | Priya Sharma | Web Development
```

Example 2: Procedure with IN and OUT Parameters

```
DELIMITER $$
CREATE PROCEDURE EnrollStudent(
    IN  p_student_id  INT,
    IN  p_course_id   INT,
    OUT p_status      VARCHAR(200)
)
BEGIN
    DECLARE v_already    INT DEFAULT 0;
    DECLARE v_price      DECIMAL(8,2) DEFAULT 0;
    DECLARE v_coursename VARCHAR(200) DEFAULT '';

    -- Check if already enrolled
    SELECT COUNT(*) INTO v_already
    FROM orders
    WHERE student_id = p_student_id
      AND course_id  = p_course_id
      AND status     = 'completed';

    IF v_already > 0 THEN
        SET p_status = CONCAT('ALREADY_ENROLLED: Already have access to this course.');
```

```
    ELSE
        -- Get course details
        SELECT price, title INTO v_price, v_coursename
        FROM courses WHERE course_id = p_course_id;

        IF v_price IS NULL THEN
            SET p_status = 'ERROR: Course not found.';
        ELSE
            -- Create the order
            INSERT INTO orders (student_id, course_id, amount_paid, status)
            VALUES (p_student_id, p_course_id, v_price, 'completed');

            SET p_status = CONCAT('SUCCESS: Enrolled in "', v_coursename, '". Paid: ₹',
v_price);
        END IF;
    END IF;
END $$
DELIMITER ;

-- Call:
CALL EnrollStudent(4, 5, @result);
SELECT @result;

-- Call again (already enrolled):
CALL EnrollStudent(4, 5, @result);
SELECT @result;
```

Output:

```
First call:
@result = 'SUCCESS: Enrolled in "MySQL Mastery". Paid: ₹1299.00'

Second call:
@result = 'ALREADY_ENROLLED: Already have access to this course.'
```

How it works: DECLARE creates local variables. SELECT ... INTO stores result in variable. OUT parameters return values to the caller via @ user variables. CALL executes the procedure.

Example 3: Procedure with WHILE LOOP — Batch Processing

```
DELIMITER $$
CREATE PROCEDURE ApplyDiscountsByTier()
BEGIN
    DECLARE v_done     INT DEFAULT 0;
    DECLARE v_id       INT;
    DECLARE v_level    VARCHAR(20);
    DECLARE v_price    DECIMAL(8,2);
    DECLARE v_discount DECIMAL(5,4);

    -- Cursor over all active courses
    DECLARE cur CURSOR FOR
        SELECT course_id, level, price FROM courses WHERE is_active = 1;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO v_id, v_level, v_price;
        IF v_done THEN LEAVE read_loop; END IF;

        -- Set discount per level
        SET v_discount = CASE v_level
            WHEN 'Beginner'     THEN 0.20
```

```
            WHEN 'Intermediate' THEN 0.10
            WHEN 'Advanced'     THEN 0.05
            ELSE 0
        END;

        -- Apply discount
        UPDATE courses
        SET price = ROUND(price * (1 - v_discount), 2)
        WHERE course_id = v_id;
    END LOOP;

    CLOSE cur;

    SELECT 'Discounts applied successfully' AS result;
END $$
DELIMITER ;

CALL ApplyDiscountsByTier();
```

Output:

```
result: Discounts applied successfully

-- courses table updated:
Beginner: -20%, Intermediate: -10%, Advanced: -5%
```

How it works: A CURSOR iterates row-by-row over a SELECT result. DECLARE CONTINUE HANDLER FOR NOT FOUND catches the "no more rows" condition. FETCH gets the next row. LEAVE exits the loop.

10. Stored Functions

⍰ Interview-Ready Definition

A stored function is like a stored procedure but MUST return exactly ONE scalar value. It can be used directly inside SQL expressions (SELECT, WHERE, ORDER BY). Functions CANNOT perform DML on permanent tables. They are DETERMINISTIC (same input → same output) or NOT DETERMINISTIC.

Example 1: Create and Use a Custom Function

```
DELIMITER $$
CREATE FUNCTION CalculateGST(p_price DECIMAL(8,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN ROUND(p_price * 1.18, 2);
END $$

CREATE FUNCTION GetPriceBracket(p_price DECIMAL(8,2))
RETURNS VARCHAR(30)
DETERMINISTIC
BEGIN
    RETURN CASE
        WHEN p_price < 1000  THEN 'Budget'
        WHEN p_price < 2000  THEN 'Standard'
        WHEN p_price < 3000  THEN 'Premium'
        ELSE 'Elite'
    END;
END $$
DELIMITER ;

-- Use functions in SELECT like built-in functions:
SELECT
  title,
  price,
  CalculateGST(price)       AS price_with_gst,
  GetPriceBracket(price)    AS bracket
FROM courses ORDER BY price DESC LIMIT 5;
```

⍰ Output:

```
title                    | price   | price_with_gst | bracket
-------------------------+---------+----------------+---------
Deep Learning            | 3499.00 | 4128.82        | Elite
Machine Learning A-Z     | 2999.00 | 3538.82        | Premium
Flutter Mobile Dev       | 2499.00 | 2948.82        | Premium
Advanced React Patterns  | 2299.00 | 2712.82        | Premium
React JS Complete Guide  | 1999.00 | 2358.82        | Standard
```

? What is the difference between a stored procedure and a stored function?

✅Stored Procedure: can perform DML (INSERT/UPDATE/DELETE), returns results via OUT params or result sets, called with CALL, cannot be used inside SELECT expression. Stored Function: MUST return one scalar value, can be used inside SELECT/WHERE/ORDER BY, CANNOT perform DML on permanent tables, called like any built-in function.

11. Transactions & ACID

Interview-Ready Definition

A transaction is a group of SQL statements executed as ONE atomic unit — either ALL succeed (COMMIT) or ALL are undone (ROLLBACK). ACID properties: Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes).

ACID Properties

Property	Meaning	EduShop Example
Atomicity	All-or-nothing — either all statements succeed or all fail	Enroll student: INSERT order AND update enrollment count. If update fails, INSERT is also undone.
Consistency	DB moves from one valid state to another valid state	Total revenue before = total revenue after any completed transaction.
Isolation	Concurrent transactions cannot see each other's incomplete changes	Two students buying the last seat — only one succeeds.
Durability	Committed changes survive crashes and power failures	After COMMIT, data is written to disk. Server restart cannot undo it.

Example 1: Basic Transaction — COMMIT and ROLLBACK

```
START TRANSACTION;

-- Step 1: Create the order
INSERT INTO orders (student_id, course_id, amount_paid, status)
VALUES (5, 2, 2999.00, 'completed');

-- Step 2: Update a summary (pretend we have one)
-- UPDATE course_stats SET enrollment_count = enrollment_count + 1 WHERE course_id = 2;

-- Step 3: Check if everything looks right
SELECT * FROM orders WHERE student_id = 5;

-- If happy:
COMMIT;    -- permanently saves all changes

-- If something went wrong at any step:
-- ROLLBACK;  -- undoes ALL changes back to before START TRANSACTION
```

Output:

```
-- After COMMIT:
Query OK, 1 row affected

-- If ROLLBACK was issued instead:
-- Order is not saved (as if it never happened)
```

Example 2: SAVEPOINT — Partial Rollback

```
START TRANSACTION;
```

```
-- Insert student 1
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Batch User 1', 'bu1@gmail.com', 'Delhi', CURDATE());

SAVEPOINT after_first_insert;   -- checkpoint!

-- Insert student 2 (might fail if email already exists)
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Batch User 2', 'bu1@gmail.com', 'Mumbai', CURDATE());  -- SAME EMAIL!
-- ERROR: Duplicate entry 'bu1@gmail.com' for key 'email'

-- Rollback ONLY to the savepoint (keeps student 1)
ROLLBACK TO SAVEPOINT after_first_insert;

-- Remove the savepoint
RELEASE SAVEPOINT after_first_insert;

-- Commit student 1 (student 2 was rolled back)
COMMIT;

SELECT * FROM students WHERE email LIKE 'bu%';
```

Output:

```
full_name      | email           | city
--------------+----------------+------
Batch User 1   | bu1@gmail.com   | Delhi
(Batch User 2 was rolled back — not saved)
```

How it works: SAVEPOINT creates a named checkpoint within a transaction. ROLLBACK TO SAVEPOINT reverts only to that point without discarding the entire transaction. RELEASE SAVEPOINT removes it.

Transaction Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	YES — can read uncommitted data	YES	YES	Fastest
READ COMMITTED	No	YES	YES	Fast
REPEATABLE READ (default InnoDB)	No	No	Mostly prevented via MVCC	Medium
SERIALIZABLE	No	No	No	Slowest

```
-- Check current isolation level
SELECT @@transaction_isolation;   -- REPEATABLE-READ (default)

-- Set for current session
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- Set globally (affects all new connections)
SET GLOBAL  TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Set for next transaction only
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
-- ... your queries ...
```

```
COMMIT;
```

❓ What is a dirty read? Give an example.

✅A dirty read occurs when Transaction A reads data that Transaction B has modified but NOT yet committed. If B then ROLLBACKs, A has read data that never officially existed. Example: B updates price from 1499 to 999 (not committed yet) → A reads price as 999 → B rolls back → the real price is still 1499, but A acted on wrong data. READ COMMITTED isolation prevents dirty reads.

❓ What is the difference between COMMIT and ROLLBACK?

✅COMMIT permanently saves all changes made since the START TRANSACTION — they survive server restarts. ROLLBACK undoes all changes back to the state at START TRANSACTION (or to a SAVEPOINT if specified). After COMMIT there is no undo. DDL commands (CREATE, ALTER, DROP) cause implicit COMMITs even inside transactions.

12. Indexes & EXPLAIN — Query Analysis

⍰ Interview-Ready Definition

An index is a separate data structure (B-Tree by default) that allows MySQL to find rows quickly without scanning every row. Indexes speed up SELECT/WHERE/JOIN dramatically but slightly slow INSERT/UPDATE/DELETE (index must be updated too).

Index Types

Type	Allows Duplicates?	NULL?	Use Case
PRIMARY KEY	No	No	Main row identifier — every table should have one
UNIQUE INDEX	No	Yes (one NULL)	Email, phone — columns that must be distinct
INDEX (non-unique)	Yes	Yes	city, level — columns used in WHERE/ORDER BY
COMPOSITE INDEX	Depends on type	Yes	(category_id, level) — multi-column conditions
FULLTEXT INDEX	Yes	Yes	title, description — full-text search with MATCH/AGAINST
SPATIAL INDEX	Yes	No	Geographic coordinate columns

Example 1: Create and Drop Indexes

```
-- Create a standard index on city (used in WHERE city = 'Hyderabad')
CREATE INDEX idx_student_city ON students(city);

-- Create composite index (used when WHERE filters BOTH columns)
CREATE INDEX idx_course_level_price ON courses(level, price);

-- Create UNIQUE index
CREATE UNIQUE INDEX idx_student_email ON students(email);

-- Create FULLTEXT index for text search
CREATE FULLTEXT INDEX idx_course_title ON courses(title);

-- View all indexes on a table
SHOW INDEX FROM courses;

-- Drop an index
DROP INDEX idx_student_city ON students;
ALTER TABLE students DROP INDEX idx_student_city;  -- alternative syntax
```

Example 2: EXPLAIN — Understand Query Execution

```
-- Always run EXPLAIN before deploying queries on large tables!

-- EXPLAIN a query
```

```
EXPLAIN SELECT full_name, email
FROM students
WHERE city = 'Hyderabad';

-- EXPLAIN a JOIN query
EXPLAIN SELECT s.full_name, c.title, o.amount_paid
FROM orders o
JOIN students s ON o.student_id = s.student_id
JOIN courses  c ON o.course_id  = c.course_id
WHERE o.status = 'completed'
ORDER BY o.amount_paid DESC;
```

Output:

```
EXPLAIN output for city search (WITHOUT index):
id | select_type | table    | type | possible_keys | key  | rows | Extra
---+-------------+----------+------+---------------+------+------+-------
1  | SIMPLE      | students | ALL  | NULL          | NULL | 10   | Using where

EXPLAIN output (AFTER CREATE INDEX idx_student_city):
id | select_type | table    | type | possible_keys    | key              | rows | Extra
---+-------------+----------+------+------------------+------------------+------+----------------
--
1  | SIMPLE      | students | ref  | idx_student_city | idx_student_city | 2    | Using index
condition
```

How it works: The EXPLAIN "type" column is the most important: ALL = full table scan (bad for large tables). ref = index used (good). const = single row by PK (best). "rows" shows estimated rows scanned — lower is better.

EXPLAIN Type Column — Best to Worst

Type	Meaning	When You See It
system	0 or 1 rows in table	Tiny lookup tables
const	Single row via PK or unique key	WHERE id = 1
eq_ref	One row from joined table per outer row	JOIN on PK
ref	Non-unique index lookup	WHERE city = 'Hyderabad'
range	Index range scan	WHERE price BETWEEN 1000 AND 2000
index	Full index scan (better than ALL)	ORDER BY indexed col
ALL	Full TABLE scan — reads every row	No usable index → slow!

Example 3: Composite Index — Left-Prefix Rule

```
-- Composite index: (level, price)
CREATE INDEX idx_level_price ON courses(level, price);

-- ✅CAN use index: filters on leftmost column (level)
EXPLAIN SELECT * FROM courses WHERE level = 'Beginner';

-- ✅CAN use index: filters on both columns in order
EXPLAIN SELECT * FROM courses WHERE level = 'Beginner' AND price < 1500;

-- ❌CANNOT use index: skips leftmost column
EXPLAIN SELECT * FROM courses WHERE price < 1500;
-- (price is the 2nd column, can't skip level — full scan!)
```

How it works: The LEFT-PREFIX RULE: a composite index (a, b, c) can be used by queries that filter on: (a), (a,b), or (a,b,c). It CANNOT be used for b alone, c alone, or (b,c) — you must start from the leftmost column.

Example 4: FULLTEXT Search

```
CREATE FULLTEXT INDEX idx_course_fulltext ON courses(title);

-- MATCH ... AGAINST: natural language search
SELECT title, MATCH(title) AGAINST ('Data Science Learning') AS relevance_score
FROM courses
WHERE MATCH(title) AGAINST ('Data Science Learning')
ORDER BY relevance_score DESC;

-- Boolean mode: more precise control
SELECT title
FROM courses
WHERE MATCH(title) AGAINST ('+Python -Excel' IN BOOLEAN MODE);
-- + = must include, - = must exclude
```

Output:

```
Natural language search:
title                   | relevance_score
------------------------+----------------
Python for Data Science | 1.2334
Deep Learning           | 0.8123
Machine Learning A-Z    | 0.6234

Boolean mode (+Python -Excel):
Python for Data Science  (has Python, no Excel)
```

? What is the difference between a clustered and non-clustered index?

✅In MySQL InnoDB, the PRIMARY KEY is always the CLUSTERED index — the actual table rows are physically stored in PK order. All other indexes are NON-CLUSTERED (secondary) — they store the index key + a pointer (the PK value) to the actual row. Looking up by secondary index = find in secondary index → get PK → look up in clustered index (two lookups if not a covering index).

? What is a covering index?

✅A covering index contains ALL columns needed by a query — MySQL can answer the query from the index alone without touching the actual table rows. Example: if a query selects only title and price, and INDEX(title, price) exists, MySQL reads only the index. Covering indexes eliminate the second lookup and dramatically speed up reads.

13. Normalization — 1NF to BCNF

⍰ Interview-Ready Definition

Normalization organises a database to reduce redundancy and improve data integrity. Each normal form eliminates a specific type of anomaly: insert, update, and delete anomalies. EduShop is already in 3NF — we show what it looked like before normalization.

Normal Forms Summary

Normal Form	Requirement	Eliminates
1NF (First)	Atomic values in each cell; no repeating groups; each row uniquely identified	Multi-valued columns, comma-separated values, duplicate rows
2NF (Second)	1NF + every non-key column fully depends on the ENTIRE primary key (not just part)	Partial dependencies (relevant when composite PK exists)
3NF (Third)	2NF + no transitive dependencies (non-key column depending on another non-key)	Transitive dependencies
BCNF (Boyce-Codd)	Stricter 3NF — every determinant must be a candidate key	Anomalies from overlapping candidate keys

Before Normalization — One Big Messy Table

order_id	student_name	student_email	student_city	course_title	course_price	category	instructor_name	instructor_email
1	Amit Joshi	amit@gmail.com	Hyd	Python Data Sci	1499	Data Science	Ravi Kumar	ravi@edushop.com
2	Amit Joshi	amit@gmail.com	Hyd	React JS	1999	Web Dev	Priya Sharma	priya@edushop.com
3	Meera Nair	meera@gmail.com	Kochi	ML A-Z	2999	Data Science	Arjun Mehta	arjun@edushop.com

⍰ **How it works:** Problems with this design: (1) UPDATE anomaly: if Amit changes email, must update in every row. (2) INSERT anomaly: cannot add a course without an order. (3) DELETE anomaly: deleting order 3 loses Meera's and Arjun's info. (4) Massive redundancy: Ravi Kumar's email repeated in every Ravi course order.

First Normal Form (1NF)

Rule: Every column must have ATOMIC (indivisible) values. No repeating groups. No arrays in a cell.

```
-- ✗VIOLATES 1NF: multiple values in one cell
CREATE TABLE orders_bad (
  order_id   INT,
  student    VARCHAR(100),
  courses    VARCHAR(500)  -- 'Python, React, ML' in one cell!
);

-- ✓1NF: one value per cell, each course in separate row
```

```sql
CREATE TABLE orders_1nf (
  order_id   INT,
  student    VARCHAR(100),
  course     VARCHAR(100)  -- one course per row
);
```

Second Normal Form (2NF)

Rule: 1NF + all non-key columns must depend on the ENTIRE composite primary key (not just part of it).

```sql
-- Suppose PK is (order_id, course_id)
-- Partial dependency: student_city depends only on student_id (part of PK)
-- This violates 2NF!

-- ❌ 2NF violation:
CREATE TABLE order_lines_bad (
  order_id     INT,
  student_id   INT,
  student_city VARCHAR(80),    -- depends only on student_id, not full PK!
  course_id    INT,
  amount_paid  DECIMAL(8,2),
  PRIMARY KEY (order_id, student_id, course_id)
);

-- ✅ 2NF: separate student info into students table
-- students(student_id, student_city)   -- city follows student
-- orders(order_id, student_id, course_id, amount_paid)  -- pure order data
```

Third Normal Form (3NF)

Rule: 2NF + no transitive dependencies (non-key column depending on another non-key column).

```sql
-- Transitive dependency example:
-- student_id → zip_code → city  (city transitively depends on student_id via zip)

-- ❌ 3NF violation:
CREATE TABLE students_bad (
  student_id INT PRIMARY KEY,
  full_name  VARCHAR(150),
  zip_code   CHAR(6),
  city       VARCHAR(80),     -- depends on zip_code, not directly on student_id!
  state      VARCHAR(50)      -- also depends on zip_code
);

-- ✅ 3NF: create separate zip_codes table
CREATE TABLE zip_codes (
  zip_code CHAR(6) PRIMARY KEY,
  city     VARCHAR(80),
  state    VARCHAR(50)
);
CREATE TABLE students_3nf (
  student_id INT PRIMARY KEY,
  full_name  VARCHAR(150),
  zip_code   CHAR(6),
  FOREIGN KEY (zip_code) REFERENCES zip_codes(zip_code)
);
```

How it works: EduShop is designed in 3NF: students table has student info. courses table has course info. categories and instructors are separate. orders links students to courses with order details only. Each piece of information lives in exactly one place.

❓ When would you INTENTIONALLY denormalize?

✅Denormalization adds back redundancy for performance. Common in: (1) Data warehouses/analytics — pre-compute JOINs for fast reporting. (2) Read-heavy systems — store derived values (total_orders count) to avoid expensive COUNT queries on every page load. (3) Search systems — duplicate data to avoid JOINs. Trade-off: faster reads, harder writes, more storage.

14. Error Handling — SIGNAL & DECLARE HANDLER

⍰ Interview-Ready Definition

MySQL error handling in stored procedures uses DECLARE HANDLER to catch errors/warnings and take action. SIGNAL raises custom errors. RESIGNAL re-raises errors. SQLSTATE codes identify specific error types.

Example 1: SIGNAL — Raise Custom Errors

```
DELIMITER $$
CREATE PROCEDURE SafeEnroll(IN p_student_id INT, IN p_course_id INT)
BEGIN
    DECLARE v_student_exists INT DEFAULT 0;
    DECLARE v_course_exists  INT DEFAULT 0;

    -- Validate student
    SELECT COUNT(*) INTO v_student_exists
    FROM students WHERE student_id = p_student_id AND is_active = 1;

    IF v_student_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Student not found or inactive.',
            MYSQL_ERRNO = 1001;
    END IF;

    -- Validate course
    SELECT COUNT(*) INTO v_course_exists
    FROM courses WHERE course_id = p_course_id AND is_active = 1;

    IF v_course_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Course not found or inactive.',
            MYSQL_ERRNO = 1002;
    END IF;

    -- Insert order if validations pass
    INSERT INTO orders (student_id, course_id, status)
    VALUES (p_student_id, p_course_id, 'completed');

    SELECT 'Enrollment successful!' AS result;
END $$
DELIMITER ;

-- Test with invalid student:
CALL SafeEnroll(999, 1);
-- ERROR 1001: Student not found or inactive.
```

Example 2: DECLARE HANDLER — Catch Errors Gracefully

```
DELIMITER $$
CREATE PROCEDURE SafeInsertStudent(
    IN p_name  VARCHAR(150),
    IN p_email VARCHAR(150)
)
BEGIN
    DECLARE v_error    INT DEFAULT 0;
    DECLARE v_errmsg   VARCHAR(200);
```

```
    -- Handler for duplicate entry error (MySQL error 1062)
    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000'
    BEGIN
        SET v_error  = 1;
        SET v_errmsg = CONCAT('Duplicate email: ', p_email, ' already exists.');
    END;

    -- Handler for any other SQL exception
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    BEGIN
        SET v_error  = 1;
        SET v_errmsg = 'An unexpected error occurred.';
    END;

    -- Try to insert
    INSERT INTO students (full_name, email, joined_date)
    VALUES (p_name, p_email, CURDATE());

    -- Check if handler was triggered
    IF v_error THEN
        SELECT v_errmsg AS error_message, 'FAILED' AS status;
    ELSE
        SELECT CONCAT('Student ', p_name, ' added successfully!') AS message, 'OK' AS
status;
    END IF;
END $$
DELIMITER ;

-- First call: works
CALL SafeInsertStudent('Test Student', 'test@gmail.com');
-- Result: Student Test Student added successfully! | OK

-- Second call: duplicate email
CALL SafeInsertStudent('Another Student', 'test@gmail.com');
-- Result: Duplicate email: test@gmail.com already exists. | FAILED
```

How it works: DECLARE HANDLER FOR SQLSTATE catches specific error codes. CONTINUE HANDLER lets execution continue after the error. EXIT HANDLER exits the procedure block. The handler can inspect and log the error then decide what to do.

15. Performance Optimization — 15 Essential Rules

⍰ Interview-Ready Definition

Query optimization is the process of writing SQL and configuring MySQL to get results faster. Small changes can turn a 10-second query into a 0.01-second query. Rule: profile first (EXPLAIN), then optimize.

The 15 Essential Optimization Rules

Rule 1: Never use SELECT * in production

SELECT * returns ALL columns including ones you don't need — wastes I/O, memory, and network. Always name specific columns.

```
-- SLOW:
SELECT * FROM orders WHERE status = 'completed';
-- FAST:
SELECT order_id, student_id, amount_paid FROM orders WHERE status = 'completed';
```

Rule 2: Avoid functions on indexed columns in WHERE

Wrapping an indexed column in a function prevents index usage — forces full table scan.

```
-- SLOW (index on joined_date is useless here):
SELECT * FROM students WHERE YEAR(joined_date) = 2022;
-- FAST (index on joined_date is used):
SELECT * FROM students WHERE joined_date >= '2022-01-01' AND joined_date < '2023-01-01';
```

Rule 3: Use LIMIT for large tables

Never load millions of rows if you display 20. Always add LIMIT to user-facing queries.

```
SELECT title, price FROM courses ORDER BY price DESC LIMIT 20;
```

Rule 4: Use indexes on JOIN columns

Columns used in ON conditions should be indexed. Without indexes, JOINs scan both tables fully.

```
-- Ensure FKs have indexes (MySQL auto-creates on FK):
CREATE INDEX idx_order_student ON orders(student_id);
CREATE INDEX idx_order_course  ON orders(course_id);
```

Rule 5: Use composite indexes correctly (left-prefix rule)

Put the most selective (highest cardinality) column first in composite indexes.

```
-- Good: status has few values; student_id has many
-- Place high-cardinality first:
CREATE INDEX idx_orders_status_student ON orders(student_id, status);
-- Query that uses it:
SELECT * FROM orders WHERE student_id = 1 AND status = 'completed';
```

Rule 6: Use EXPLAIN before deploying any complex query

Run EXPLAIN to see if MySQL uses indexes or does full scans. Fix before going to production.

```
EXPLAIN SELECT s.full_name, COUNT(o.order_id)
FROM students s
JOIN orders o ON s.student_id = o.student_id
GROUP BY s.student_id;
-- Check 'type' column: ALL = bad, ref/eq_ref = good
```

Rule 7: Avoid LIKE with leading wildcard

LIKE '%word' cannot use a B-Tree index — full scan. Use FULLTEXT index for text search.

```
-- SLOW (full scan):
SELECT title FROM courses WHERE title LIKE '%Python%';
-- FAST (FULLTEXT):
SELECT title FROM courses
WHERE MATCH(title) AGAINST ('Python' IN BOOLEAN MODE);
```

Rule 8: Use NOT EXISTS instead of NOT IN for subqueries

NOT IN with a subquery that can return NULLs returns empty result. NOT EXISTS is NULL-safe and often faster.

```
-- RISKY (empty if any student_id is NULL):
SELECT * FROM students WHERE student_id NOT IN (SELECT student_id FROM orders);
-- SAFE & FAST:
SELECT s.* FROM students s
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.student_id = s.student_id);
```

Rule 9: Use keyset pagination instead of OFFSET for large pages

OFFSET 100000 scans and discards 100,000 rows. Keyset uses index directly.

```
-- SLOW (OFFSET 1000000):
SELECT * FROM courses ORDER BY course_id LIMIT 20 OFFSET 1000000;
-- FAST (keyset):
SELECT * FROM courses WHERE course_id > :last_seen_id ORDER BY course_id LIMIT 20;
```

Rule 10: Use covering indexes for read-heavy queries

A covering index contains all columns the query needs — no table lookup required.

```
-- Query only needs student_id and status:
CREATE INDEX idx_orders_covering ON orders(student_id, status, amount_paid);
-- MySQL answers from index alone (no table access):
SELECT student_id, status, amount_paid FROM orders WHERE student_id = 1;
```

Rule 11: Use connection pooling

Opening a new DB connection for every request is expensive. Use a connection pool.

```
-- In application: use connection pool libraries
-- Node.js: mysql2/promise pool
-- Java: HikariCP
-- Python: SQLAlchemy pool
-- Do NOT create new connection per request in production
```

Rule 12: Use batch INSERT instead of row-by-row

Multi-row INSERT is 10-100x faster than individual INSERTs.

```
-- SLOW (10 round trips):
INSERT INTO t VALUES (1); INSERT INTO t VALUES (2); ...
-- FAST (1 round trip):
INSERT INTO t VALUES (1), (2), (3), (4), (5), (6), (7), (8), (9), (10);
```

Rule 13: Archive old data

Tables with 100M rows are slow. Move data older than N years to archive tables.

```
-- Create archive table:
CREATE TABLE orders_archive LIKE orders;
-- Move old data:
INSERT INTO orders_archive SELECT * FROM orders WHERE order_date < '2020-01-01';
DELETE FROM orders WHERE order_date < '2020-01-01';
```

Rule 14: Use appropriate data types

TINYINT vs INT vs BIGINT — smaller type = less storage = faster comparison.

```
-- BAD: VARCHAR(255) for a 2-char country code:
country CHAR(2)  -- 2 bytes, not VARCHAR(255)!
-- BAD: INT for a 0/1 flag:
is_active TINYINT(1)  -- 1 byte, not INT (4 bytes)
```

Rule 15: Monitor with slow query log

Enable slow query log to find queries that take too long in production.

```
-- Enable slow query log:
SET GLOBAL slow_query_log = 1;
SET GLOBAL long_query_time = 1;  -- log queries > 1 second
SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';
-- View slow queries:
SHOW VARIABLES LIKE 'slow_query_log%';
```

SECTION A — 30 Master Interview Questions

Covering all advanced MySQL concepts from Part 2. These are the most-asked questions for fresher and mid-level MySQL developer roles.

❓ What is a window function and how is it different from GROUP BY?

✅Window functions calculate across a related set of rows (a "window") WITHOUT collapsing them into one row. GROUP BY collapses rows — you lose individual detail. Example: SELECT title, price, AVG(price) OVER (PARTITION BY category_id) AS cat_avg FROM courses — shows each course row WITH its category's average price. GROUP BY would give ONE row per category.

❓ Explain ROW_NUMBER, RANK, and DENSE_RANK with an example.

✅All three rank rows in ordered windows. Given prices 3499, 2999, 2999, 1999: ROW_NUMBER() = 1,2,3,4 (always unique). RANK() = 1,2,2,4 (same rank for ties, skips next). DENSE_RANK() = 1,2,2,3 (same rank for ties, no skip). Use DENSE_RANK when you want "top N positions without gaps."

❓ What is the difference between LAG() and LEAD()?

✅LAG(col, n) accesses the value n rows BEFORE the current row within the partition. LEAD(col, n) accesses n rows AFTER. Both return NULL at the boundary. Usage: LAG to compare current vs previous (month-over-month change). LEAD to calculate "days until next order." Both accept a default for boundary NULLs: LAG(price, 1, 0).

❓ What is a CTE? When would you use it over a subquery?

✅A CTE (Common Table Expression) is a named temporary result set defined with WITH. Use a CTE over a subquery when: (1) You need to reference the same result multiple times in the query. (2) The query logic has multiple logical steps that benefit from named intermediate results. (3) You need recursion (WITH RECURSIVE). CTEs make complex queries more readable and maintainable.

❓ What is a recursive CTE? Give a use case.

✅A recursive CTE calls itself repeatedly until a termination condition is met. Use cases: (1) Org chart traversal — find all subordinates of a manager. (2) Category hierarchy — find all subcategories. (3) Generate date/number series. Syntax: WITH RECURSIVE cte AS (anchor UNION ALL recursive_part WHERE stop_condition).

❓ What is a view? What are its advantages and disadvantages?

✅A view is a saved SELECT query that acts as a virtual table. Advantages: simplifies complex queries, implements security (hide sensitive columns), provides stable interface when underlying schema changes. Disadvantages: no data storage (re-executes every time), complex views cannot be updated, no indexes on view columns (base table indexes still used).

❓ What is the difference between a view and a materialized view?

✅A regular view re-executes its query every time it is accessed — always fresh but potentially slow. A materialized view stores the query results physically — fast reads but potentially stale. MySQL does NOT natively support materialized views. Simulate with a periodic-refresh summary table + scheduled EVENT or TRIGGER.

❓ Explain BEFORE vs AFTER triggers with examples.

✅BEFORE triggers fire BEFORE the row is saved. Can MODIFY NEW values and ABORT the operation with SIGNAL. Use for validation. Example: BEFORE UPDATE prevents price reduction > 50%. AFTER triggers fire AFTER the row is saved. Cannot modify values. Used for side effects: audit logging, updating derived tables. Example: AFTER INSERT on orders → update course enrollment count.

❓ What are ACID properties and why do they matter?

✅Atomicity: all operations in a transaction succeed or all fail — no partial saves. Consistency: database moves from one valid state to another — constraints always satisfied. Isolation: concurrent transactions don't see each other's incomplete work. Durability: committed data survives system crashes. Without ACID: a payment debit could succeed while credit fails; two users could book the same seat.

❓ What are transaction isolation levels and what problems do they prevent?

✅READ UNCOMMITTED: allows dirty reads (read uncommitted data). READ COMMITTED: prevents dirty reads. REPEATABLE READ (MySQL default): also prevents non-repeatable reads (same row gives different values within transaction). SERIALIZABLE: also prevents phantom reads (new rows appearing in range scans). Higher isolation = fewer anomalies but lower concurrency.

❓ What is an index? What are the downsides of too many indexes?

✅An index is a B-Tree data structure that speeds up data lookup — like a book index. Downsides of over-indexing: (1) Each index consumes disk space. (2) INSERT/UPDATE/DELETE become slower — all indexes must be updated. (3) Query optimizer may choose a suboptimal index if too many exist. Index only columns used in WHERE, JOIN ON, and ORDER BY.

❓ Explain the composite index left-prefix rule.

✅A composite index on (a, b, c) can be used by queries filtering on: a alone, (a, b), or (a, b, c). It CANNOT be used for b alone, c alone, (b, c), or (a, c) — you must start from the leftmost column and proceed in order. Put the most selective column (highest cardinality) leftmost for best performance.

❓ What is a covering index?

✅A covering index contains ALL columns needed by a query. MySQL can answer the query entirely from the index without touching the actual table rows (avoiding the "table lookup" step). Example: INDEX(student_id, status, amount_paid) covers SELECT student_id, amount_paid FROM orders WHERE student_id=1 AND status='completed'. EXPLAIN shows "Using index" in Extra column.

❓ What is the difference between CHAR, VARCHAR, and TEXT?

✅CHAR(n): fixed-length, always uses n bytes (pads with spaces), fast for fixed data. VARCHAR(n): variable-length, uses actual bytes + 1-2 overhead, space-efficient. TEXT: variable unlimited size, cannot have DEFAULT value, cannot be fully indexed, stored separately from row. Use CHAR for codes, VARCHAR for names/emails, TEXT for articles/descriptions.

❓ What is normalization and what are the normal forms?

✅1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependencies on composite PK. 3NF: 2NF + no transitive dependencies (non-key → non-key). BCNF: every determinant is a candidate key. Most production databases target 3NF — balances integrity and performance.

❓ What is denormalization and when is it appropriate?

✅Denormalization intentionally adds redundancy for performance. Appropriate in: analytics/data warehouses (pre-compute joins), read-heavy systems (cache derived values like total_orders to avoid COUNT queries), search systems, and microservices (local copies of foreign data to avoid cross-service calls). Trade-off: faster reads, more complex writes, potential staleness.

❓ What is SIGNAL in MySQL stored procedures?

✅SIGNAL SQLSTATE 'code' raises a custom error from within a stored procedure or trigger. SQLSTATE '45000' is the generic user-defined error code. You can set MESSAGE_TEXT, MYSQL_ERRNO, and other properties. It aborts the current statement and propagates the error to the caller. Use it for business rule validation inside stored objects.

❓ What is a stored procedure cursor and when is it used?

✅A cursor allows row-by-row processing of a SELECT result inside a stored procedure — like a pointer that moves through rows. Use only when SET-based operations (UPDATE, INSERT ... SELECT) cannot solve the problem — row-by-row is inherently slower. Common for: complex business logic per row, batch processing with per-row decisions, tree traversal.

❓ What are the string functions you use most frequently?

✅CONCAT/CONCAT_WS for joining strings. UPPER/LOWER for case. TRIM/LTRIM/RTRIM for cleaning spaces. SUBSTRING/LEFT/RIGHT for extraction. REPLACE for find-and-replace. INSTR/LOCATE for position finding. LPAD/RPAD for formatting. LENGTH/CHAR_LENGTH for size. REGEXP_REPLACE for pattern replacement. FORMAT for number display.

❓ How do you find the second highest salary/price in MySQL?

✅Multiple approaches: (1) Subquery: SELECT MAX(price) FROM courses WHERE price < (SELECT MAX(price) FROM courses). (2) ORDER BY + LIMIT: SELECT DISTINCT price FROM courses ORDER BY price DESC LIMIT 1 OFFSET 1. (3) Window function (most robust): SELECT price FROM (SELECT price, DENSE_RANK() OVER (ORDER BY price DESC) AS rnk FROM courses) r WHERE rnk = 2.

❓ What is EXPLAIN and what do you look for in its output?

✅EXPLAIN shows MySQL's query execution plan — how it will fetch data. Key columns: type (ALL=full scan=bad, ref/const=index=good), key (which index used), rows (estimated rows scanned — lower is better), Extra (Using filesort/temporary = potential performance issue; Using index = covering index = fast).

❓ How do you find duplicate rows in a table?

✅SELECT email, COUNT(*) AS cnt FROM students GROUP BY email HAVING COUNT(*) > 1. To see all duplicate rows: SELECT * FROM students WHERE email IN (SELECT email FROM students GROUP BY email HAVING COUNT(*) > 1). To delete duplicates keeping the lowest ID: DELETE s1 FROM students s1 JOIN students s2 ON s1.email=s2.email AND s1.student_id > s2.student_id.

❓ What is the difference between UNION and JOIN?

✅JOIN combines columns — adds more columns from related tables (horizontal combination). UNION combines rows — stacks result sets from separate queries (vertical combination). JOIN requires a related key. UNION requires same column count and compatible types. JOIN connects tables with relationships; UNION combines similar result sets from different queries.

❓ How does MySQL handle NULL in GROUP BY?

✅In GROUP BY, all NULL values are grouped together into one group (NULLs are treated as equal for grouping purposes). Example: if city has NULLs, GROUP BY city creates one group for all NULL cities. In WHERE, ORDER BY, and HAVING, NULLs are generally excluded or sort to one end.

❓ What is the difference between IFNULL(), NULLIF(), and COALESCE()?

✅IFNULL(a, b): if a is NULL return b, else return a (2 args). COALESCE(a,b,c,...): returns first non-NULL from list (any number of args, SQL standard). NULLIF(a, b): returns NULL if a=b, else returns a. Use case: NULLIF(divisor, 0) prevents division by zero. COALESCE is the most versatile and portable.

❓ What is ROW_NUMBER() and what is it used for?

✅ROW_NUMBER() assigns a unique sequential integer to each row within a partition ordered by specified columns. Always unique — no ties. Common uses: (1) Pagination — WHERE row_num BETWEEN 1 AND 10. (2) Deduplication — keep only the latest row per group: WHERE row_num = 1 (PARTITION BY key ORDER BY date DESC). (3) Generate unique serial numbers within groups.

❓ How do you calculate a running total in SQL?

✅Use SUM() OVER with a window frame: SELECT date, amount, SUM(amount) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM orders. The ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW frame means "from the first row to the current row" — this grows as you move down rows.

❓ What is a derived table and how is it different from a CTE?

✅A derived table is a subquery in the FROM clause with an alias — it acts as a virtual table for that query. A CTE does the same thing but is defined BEFORE the main query with WITH, has a name, and can be referenced MULTIPLE TIMES. CTEs are more readable and support recursion. Derived tables are inline and limited to one use.

❓ What is the difference between an INNER JOIN and a subquery with IN?

✅Both can produce the same results. INNER JOIN combines rows from both tables — useful when you need columns from both. Subquery with IN filters one table based on values from another — cleaner when you only need one table's columns. JOINs are generally more efficient because the optimizer can use indexes on both sides. EXISTS is often faster than IN for large subquery results.

❓ How do you optimize a slow query? Walk through your process.

✅(1) Run EXPLAIN to see the execution plan. (2) Look for type=ALL (full scan) on large tables. (3) Check if appropriate indexes exist — add missing ones. (4) Verify WHERE conditions on indexed columns don't wrap columns in functions. (5) Check for N+1 query issues (correlated subqueries per row). (6) Consider rewriting with CTEs or window functions. (7) Check if LIMIT is used. (8) Enable slow query log for ongoing monitoring.

SECTION B — 15 Coding Challenges (Multi-Approach)

For each challenge, multiple SQL approaches are shown — including which is the most optimised and why. This is how interviewers test deep understanding.

CQ1: Find the 2nd Highest Priced Course

Challenge: Return the title and price of the second most expensive course.

Approach 1 — Subquery (Classic):

```
SELECT title, price FROM courses
WHERE price = (
    SELECT MAX(price) FROM courses
    WHERE price < (SELECT MAX(price) FROM courses)
);
```

⚡ Alternative — Works but runs subquery twice

Runs MAX twice. If there are ties at the top, returns NULL incorrectly.

Approach 2 — ORDER BY + LIMIT + OFFSET:

```
SELECT DISTINCT title, price FROM courses
ORDER BY price DESC
LIMIT 1 OFFSET 1;
```

⚡ Alternative — Simple but DISTINCT required for tie handling

Approach 3 — Window Function DENSE_RANK (Best):

```
SELECT title, price
FROM (
    SELECT title, price,
           DENSE_RANK() OVER (ORDER BY price DESC) AS rnk
    FROM courses
) ranked
WHERE rnk = 2;
```

✅ BEST — Handles ties correctly, single pass, scalable to Nth highest

DENSE_RANK correctly handles ties (no gaps). Change WHERE rnk = 2 to rnk = N for Nth highest. Runs once.

✅ Output:

```
title: Machine Learning A-Z  |  price: 2999.00
```

CQ2: Find Students Who Never Placed an Order

Challenge: List all students who have not placed any order.

Approach 1 — NOT IN (Dangerous!):

```
SELECT full_name FROM students
WHERE student_id NOT IN (SELECT student_id FROM orders);
```

❌ DANGEROUS — Returns EMPTY if any student_id in orders is NULL

Approach 2 — LEFT JOIN + IS NULL:

```
SELECT s.full_name
FROM students s
LEFT JOIN orders o ON s.student_id = o.student_id
WHERE o.order_id IS NULL;
```

✅ BEST — Efficient, NULL-safe, uses index on student_id

LEFT JOIN keeps all students; WHERE o.order_id IS NULL keeps only unmatched ones. Uses FK index.

Approach 3 — NOT EXISTS (Safe, Equivalent to LEFT JOIN):

```
SELECT full_name FROM students s
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.student_id = s.student_id);
```

✅ EQUALLY GOOD — NULL-safe, readable, efficient with index

✅ Output:

```
(In sample data all students have orders — result would be empty)

If a student had no orders, their name would appear here.
```

CQ3: Top 3 Revenue-Generating Courses

Challenge: Find the top 3 courses by total revenue (completed orders only).

Approach 1 — GROUP BY + ORDER BY + LIMIT (Standard):

```
SELECT c.title, SUM(o.amount_paid) AS total_revenue
FROM orders o
JOIN courses c ON o.course_id = c.course_id
WHERE o.status = 'completed'
GROUP BY c.course_id, c.title
ORDER BY total_revenue DESC
LIMIT 3;
```

✅ BEST — Simple, efficient, uses GROUP BY + aggregation

Approach 2 — CTE + Window Function (When ties matter):

```
WITH course_revenue AS (
    SELECT c.title, SUM(o.amount_paid) AS total_revenue
    FROM orders o JOIN courses c ON o.course_id = c.course_id
    WHERE o.status = 'completed'
    GROUP BY c.course_id, c.title
)
SELECT title, total_revenue
FROM (
    SELECT title, total_revenue,
           DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS rnk
    FROM course_revenue
) ranked
WHERE rnk <= 3;
```

⚡ Alternative — Use when ties exist (both rank-3 courses should appear)

Approach 1 with LIMIT cuts off tied 3rd-place courses. DENSE_RANK includes all ties.

✅ Output:

```
title                  | total_revenue
----------------------+-------------
Machine Learning A-Z   | 5998.00
Python for Data Sci    | 5996.00
Deep Learning          | 3499.00
```

CQ4: Month-over-Month Revenue Growth

Challenge: Calculate monthly revenue AND the % change compared to the previous month.

Approach 1 — Window Function LAG (Best):

```
WITH monthly AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m')   AS year_month,
        SUM(amount_paid)                   AS revenue
```

```
    FROM orders WHERE status = 'completed'
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    year_month,
    revenue,
    LAG(revenue) OVER (ORDER BY year_month)  AS prev_month_rev,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY year_month))
        / LAG(revenue) OVER (ORDER BY year_month) * 100
    , 2)                                     AS pct_change
FROM monthly
ORDER BY year_month;
```

⭐ BEST — Clean, single pass, handles NULL naturally for first month

📤 Output:

```
year_month | revenue | prev_month_rev | pct_change
-----------+---------+----------------+-----------
2021-01    | 3499.00 | NULL           | NULL
2021-09    | 1999.00 | 3499.00        | -42.84
2021-12    | 1499.00 | 1999.00        | -25.01
2022-02    | 1499.00 | 1499.00        | 0.00
2022-04    | 1799.00 | 1499.00        | 20.01
2022-05    | 2999.00 | 1799.00        | 66.70
...
```

Approach 2 — Self-Join (Old way, less clean): JOIN monthly m1 ON m2.year_month = DATE_FORMAT(DATE_ADD(STR_TO_DATE(CONCAT(m1.year_month,'-01'),'%Y-%m-%d'), INTERVAL 1 MONTH), '%Y-%m'). Messy date arithmetic, harder to read.

CQ5: Find Duplicate Emails in Students Table

Challenge: Find all emails that appear more than once, and show all student records for those emails.

Approach 1 — GROUP BY + HAVING (Find duplicate emails):

```
SELECT email, COUNT(*) AS cnt
FROM students GROUP BY email HAVING COUNT(*) > 1;
```

⚡ Alternative — Shows just the email and count, not full rows

Approach 2 — CTE + JOIN (Show all duplicate rows with details):

```
WITH dupes AS (
    SELECT email FROM students
    GROUP BY email HAVING COUNT(*) > 1
)
SELECT s.*
FROM students s JOIN dupes ON s.email = dupes.email
ORDER BY s.email, s.student_id;
```

⭐ BEST — Shows full details of all duplicate rows

Approach 3 — Delete duplicates (keep lowest ID):

```
DELETE s1 FROM students s1
JOIN students s2
  ON s1.email = s2.email AND s1.student_id > s2.student_id;
-- Keeps the row with the smallest student_id for each email
```

⭐ BEST for cleanup — Self-join delete, keeps original (lowest ID)

CQ6: Running Total of Revenue by Student

Challenge: For each order, show the cumulative amount spent by that student up to and including that order.

Approach 1 — Correlated Subquery (Works but SLOW):

```
SELECT
  o1.student_id,
  o1.order_date,
  o1.amount_paid,
  (SELECT SUM(o2.amount_paid) FROM orders o2
   WHERE o2.student_id = o1.student_id
     AND o2.order_date <= o1.order_date
     AND o2.status = 'completed') AS running_total
FROM orders o1
WHERE o1.status = 'completed'
ORDER BY o1.student_id, o1.order_date;
```

⚡ **Alternative — Runs subquery once per row = O(n²) — slow on large tables**

Approach 2 — Window Function SUM OVER (Best):

```
SELECT
  s.full_name,
  o.order_date,
  o.amount_paid,
  SUM(o.amount_paid) OVER (
      PARTITION BY o.student_id
      ORDER BY o.order_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total_per_student
FROM orders o
JOIN students s ON o.student_id = s.student_id
WHERE o.status = 'completed'
ORDER BY s.full_name, o.order_date;
```

✅ **BEST — Single pass O(n), uses window frame, handles partitioning by student**

✅ **Output:**

```
full_name    | order_date          | amount_paid | running_total
-------------+---------------------+-------------+--------------
Amit Joshi   | 2022-02-10 10:00:00 | 1499.00     | 1499.00
Amit Joshi   | 2022-04-05 11:30:00 | 1799.00     | 3298.00
Kavya Reddy  | 2023-01-15 11:00:00 | 1499.00     | 1499.00
Kavya Reddy  | 2023-02-20 13:00:00 | 2999.00     | 4498.00
Meera Nair   | 2022-05-01 09:00:00 | 2999.00     | 2999.00
Meera Nair   | 2023-02-14 15:00:00 | 1299.00     | 4298.00
```

CQ7: Courses Above Category Average Price

Challenge: Find all courses priced above the average price of their own category.

Approach 1 — Correlated Subquery (Classic):

```
SELECT title, price, category_id
FROM courses c
WHERE price > (SELECT AVG(price) FROM courses WHERE category_id = c.category_id);
```

⚡ **Alternative — Correct but runs subquery once per row**

Approach 2 — CTE (Readable):

```
WITH cat_avg AS (
    SELECT category_id, AVG(price) AS avg_price
    FROM courses GROUP BY category_id
)
SELECT c.title, c.price, ca.avg_price
FROM courses c
JOIN cat_avg ca ON c.category_id = ca.category_id
WHERE c.price > ca.avg_price;
```

✅ **BEST — Computes averages once, then JOIN — most efficient**

Approach 3 — Window Function (Elegant):

```
SELECT title, price, category_id, cat_avg
FROM (
    SELECT title, price, category_id,
           AVG(price) OVER (PARTITION BY category_id) AS cat_avg
    FROM courses
) x
WHERE price > cat_avg;
```

✅ EQUALLY GOOD — Window function computes avg once per partition

✅ Output:

```
title                    | price    | cat_avg
-------------------------+----------+---------
Machine Learning A-Z     | 2999.00 | 2665.67
Deep Learning            | 3499.00 | 2665.67
Advanced React Patterns  | 2299.00 | 1865.67
Business Analytics       | 1799.00 | 1249.00
Flutter Mobile Dev       | 2499.00 | 2499.00
```

CQ8: Students Who Bought ALL Advanced Courses

Challenge: Find students who have purchased EVERY Advanced-level course (relational division).

Approach 1 — COUNT Trick (Classic Relational Division):

```
-- First, how many Advanced courses exist?
-- SELECT COUNT(*) FROM courses WHERE level = 'Advanced'  → 4

SELECT s.full_name
FROM students s
JOIN orders o ON s.student_id = o.student_id
JOIN courses c ON o.course_id = c.course_id
WHERE c.level = 'Advanced' AND o.status = 'completed'
GROUP BY s.student_id, s.full_name
HAVING COUNT(DISTINCT c.course_id) = (
    SELECT COUNT(*) FROM courses WHERE level = 'Advanced'
);
```

✅ BEST — Efficient: COUNT DISTINCT bought = total available

✅ Output:

```
(In sample data, no student has bought all 4 Advanced courses — result is empty)

If Kavya had bought all 4 Advanced courses:
full_name: Kavya Reddy
```

CQ9: Rank Instructors by Average Course Rating

Challenge: Rank instructors by the average rating of their courses, showing rank, name, courses count, and avg rating.

Approach 1 — CTE + Window Function (Best):

```
WITH instructor_stats AS (
    SELECT
        i.instructor_id,
        i.full_name,
        COUNT(DISTINCT c.course_id)    AS course_count,
        ROUND(AVG(r.rating), 2)        AS avg_rating,
        COUNT(r.review_id)             AS total_reviews
    FROM instructors i
    LEFT JOIN courses c ON i.instructor_id = c.instructor_id
    LEFT JOIN reviews r ON c.course_id = r.course_id
```

```
    GROUP BY i.instructor_id, i.full_name
)
SELECT
    RANK() OVER (ORDER BY avg_rating DESC) AS ranking,
    full_name,
    course_count,
    avg_rating,
    total_reviews
FROM instructor_stats
ORDER BY ranking;
```

⍰ BEST — CTE separates aggregation; window function adds rank cleanly

⍰ Output:

```
ranking | full_name      | course_count | avg_rating | total_reviews
--------+---------------+--------------+------------+--------------
1       | Arjun Mehta    | 3            | 4.67       | 3
2       | Ravi Kumar     | 2            | 4.67       | 4
3       | Priya Sharma   | 3            | 4.50       | 2
4       | Sneha Reddy    | 2            | NULL       | 0
4       | Vikram Singh   | 0            | NULL       | 0
```

CQ10: Find Courses Not Purchased in the Last 6 Months

Challenge: Find active courses that have had no completed purchase in the last 6 months.

Approach 1 — NOT EXISTS (Recommended):

```
SELECT c.title, c.price, c.level
FROM courses c
WHERE c.is_active = 1
  AND NOT EXISTS (
      SELECT 1 FROM orders o
      WHERE o.course_id = c.course_id
        AND o.status = 'completed'
        AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
  )
ORDER BY c.price DESC;
```

⍰ BEST — NULL-safe, short-circuits at first found row

Approach 2 — LEFT JOIN + IS NULL:

```
SELECT c.title, c.price
FROM courses c
LEFT JOIN orders o ON c.course_id = o.course_id
  AND o.status = 'completed'
  AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
WHERE c.is_active = 1 AND o.order_id IS NULL;
```

⍰ EQUALLY GOOD — Anti-join pattern, efficient with proper indexes

CQ11: Create a Pivot Report — Revenue by Level and Quarter

Challenge: Show revenue as a pivot: rows = course level, columns = Q1/Q2/Q3/Q4.

Approach 1 — Conditional SUM (CASE in SUM):

```
SELECT
  c.level,
  SUM(CASE WHEN QUARTER(o.order_date)=1 THEN o.amount_paid ELSE 0 END) AS Q1,
  SUM(CASE WHEN QUARTER(o.order_date)=2 THEN o.amount_paid ELSE 0 END) AS Q2,
  SUM(CASE WHEN QUARTER(o.order_date)=3 THEN o.amount_paid ELSE 0 END) AS Q3,
  SUM(CASE WHEN QUARTER(o.order_date)=4 THEN o.amount_paid ELSE 0 END) AS Q4,
  SUM(o.amount_paid)                                                    AS total
FROM orders o
JOIN courses c ON o.course_id = c.course_id
```

```
WHERE o.status = 'completed'
GROUP BY c.level
ORDER BY c.level;
```

✅ BEST — Standard SQL pivot pattern, one pass, highly readable

✅ Output:

```
level        | Q1      | Q2      | Q3      | Q4      | total
-------------+---------+---------+---------+---------+---------
Advanced     | 7996.00 | 2299.00 | 2499.00 | 0.00    | 12794.00
Beginner     | 2498.00 | 0.00    | 699.00  | 1499.00 | 4696.00
Intermediate | 1799.00 | 3998.00 | 0.00    | 1299.00 | 7096.00
```

CQ12: Find Students Who Bought Courses in Multiple Categories

Challenge: Find students who have bought courses from 2 or more different categories.

```
SELECT
    s.full_name,
    COUNT(DISTINCT cat.category_id)    AS categories_purchased,
    GROUP_CONCAT(DISTINCT cat.category_name ORDER BY cat.category_name) AS categories
FROM students s
JOIN orders o ON s.student_id = o.student_id
JOIN courses c ON o.course_id = c.course_id
JOIN categories cat ON c.category_id = cat.category_id
WHERE o.status = 'completed'
GROUP BY s.student_id, s.full_name
HAVING COUNT(DISTINCT cat.category_id) >= 2
ORDER BY categories_purchased DESC;
```

✅ BEST — GROUP_CONCAT lists categories; HAVING filters; single pass

✅ Output:

```
full_name    | categories_purchased | categories
-------------+----------------------+---------------------------------
Amit Joshi   | 2                    | Data Science, Web Development
Meera Nair   | 2                    | Data Science, Web Development
Sanjay Patel | 2                    | Business
```

CQ13: Find First Purchase Date for Each Student

Challenge: For each student, find their first order date and the course they bought.

Approach 1 — Correlated Subquery (Classic):

```
SELECT s.full_name, o.order_date, c.title
FROM orders o
JOIN students s ON o.student_id = s.student_id
JOIN courses  c ON o.course_id  = c.course_id
WHERE o.order_date = (
    SELECT MIN(o2.order_date) FROM orders o2
    WHERE o2.student_id = o.student_id
);
```

⚡ Alternative — Correct but correlated subquery runs per row

Approach 2 — CTE + ROW_NUMBER (Best):

```
WITH ranked_orders AS (
    SELECT
        o.*,
        ROW_NUMBER() OVER (PARTITION BY o.student_id ORDER BY o.order_date ASC) AS rn
    FROM orders o
)
SELECT s.full_name, ro.order_date AS first_order_date, c.title AS first_course
FROM ranked_orders ro
```

```
JOIN students s ON ro.student_id = s.student_id
JOIN courses  c ON ro.course_id  = c.course_id
WHERE ro.rn = 1
ORDER BY s.full_name;
```

⍰ BEST — Window function assigns row 1 to first order per student, single pass

⍰ Output:

```
full_name     | first_order_date    | first_course
--------------+---------------------+------------------------
Amit Joshi    | 2022-02-10 10:00:00 | Python for Data Science
Anjali Singh  | 2023-03-10 10:00:00 | Flutter Mobile Dev
Deepak Verma  | 2021-09-10 10:00:00 | React JS Complete Guide
Kavya Reddy   | 2023-01-15 11:00:00 | Python for Data Science
Meera Nair    | 2022-05-01 09:00:00 | Machine Learning A-Z
...
```

CQ14: Calculate Percentile Rank of Each Course

Challenge: Show each course with its price percentile (what % of courses are cheaper).

```
SELECT
  title,
  price,
  -- PERCENT_RANK: (rank-1)/(total-1) -- gives 0 for min, 1 for max
  ROUND(PERCENT_RANK() OVER (ORDER BY price) * 100, 1)   AS percentile_pct,
  -- CUME_DIST: proportion of rows <= current value
  ROUND(CUME_DIST()    OVER (ORDER BY price) * 100, 1)   AS cumulative_dist_pct,
  -- NTILE(100) for true percentile bucket
  NTILE(4) OVER (ORDER BY price)                         AS quartile
FROM courses
ORDER BY price;
```

⍰ BEST — Built-in PERCENT_RANK and CUME_DIST window functions

⍰ Output:

```
title                     | price   | percentile_pct | cumulative_dist | quartile
--------------------------+---------+----------------+-----------------+---------
Excel for Business        | 699.00  | 0.0            | 10.0            | 1
UI/UX Design Fund.        | 999.00  | 11.1           | 20.0            | 1
MySQL Mastery             | 1299.00 | 22.2           | 30.0            | 1
Python for Data Science   | 1499.00 | 33.3           | 40.0            | 2
Business Analytics        | 1799.00 | 44.4           | 50.0            | 2
React JS Complete Guide   | 1999.00 | 55.6           | 60.0            | 2
Advanced React Patterns   | 2299.00 | 66.7           | 70.0            | 3
Flutter Mobile Dev        | 2499.00 | 77.8           | 80.0            | 3
Machine Learning A-Z      | 2999.00 | 88.9           | 90.0            | 4
Deep Learning             | 3499.00 | 100.0          | 100.0           | 4
```

CQ15: Build a Full Student Dashboard Report

Challenge: For each student show: total orders, total spent, average rating given, favourite category, first and last order date — all in ONE query.

```
WITH
order_summary AS (
    SELECT
        o.student_id,
        COUNT(*)                        AS total_orders,
        SUM(o.amount_paid)              AS total_spent,
        MIN(DATE(o.order_date))         AS first_order,
        MAX(DATE(o.order_date))         AS last_order
    FROM orders o WHERE o.status = 'completed'
    GROUP BY o.student_id
),
```

```
review_summary AS (
    SELECT student_id, ROUND(AVG(rating),2) AS avg_rating_given
    FROM reviews GROUP BY student_id
),
fav_category AS (
    SELECT student_id, category_name,
           RANK() OVER (PARTITION BY student_id ORDER BY cnt DESC) AS rnk
    FROM (
        SELECT o.student_id, cat.category_name, COUNT(*) AS cnt
        FROM orders o
        JOIN courses c ON o.course_id = c.course_id
        JOIN categories cat ON c.category_id = cat.category_id
        WHERE o.status = 'completed'
        GROUP BY o.student_id, cat.category_name
    ) x
)
SELECT
    s.full_name,
    IFNULL(os.total_orders, 0)   AS total_orders,
    IFNULL(os.total_spent, 0)    AS total_spent,
    rs.avg_rating_given,
    fc.category_name             AS fav_category,
    os.first_order,
    os.last_order
FROM students s
LEFT JOIN order_summary os  ON s.student_id = os.student_id
LEFT JOIN review_summary rs ON s.student_id = rs.student_id
LEFT JOIN fav_category fc   ON s.student_id = fc.student_id AND fc.rnk = 1
ORDER BY total_spent DESC;
```

⭐ BEST — Multi-CTE pipeline, each concern separated, single final JOIN

⭐ Output:

```
full_name      | total_orders | total_spent | avg_rating | fav_category | first_order | last_order
--------------+-------------+------------+-----------+-------------+------------+----------
-
Kavya Reddy    | 2            | 4498.00     | 5.00       | Data Science | 2023-01-15  | 2023-02-20
Meera Nair     | 2            | 4298.00     | 5.00       | Data Science | 2022-05-01  | 2023-02-14
Rohit Sharma   | 1            | 3499.00     | 4.00       | Data Science | 2021-01-15  | 2021-01-15
Amit Joshi     | 2            | 3298.00     | 4.50       | Data Science | 2022-02-10  | 2022-04-05
Anjali Singh   | 1            | 2499.00     | NULL       | Mobile Dev   | 2023-03-10  | 2023-03-10
Deepak Verma   | 1            | 1999.00     | NULL       | Web Dev      | 2021-09-10  | 2021-09-10
...
```

This query demonstrates mastery of CTEs, window functions, conditional aggregates, and LEFT JOINs — exactly the kind of complex query senior developers write in production reporting systems.

Part 2 Complete — Study Recap

⭐ What You Have Covered Across Parts 1 & 2

Part 1 Fundamentals:

- Data Types, CREATE DATABASE/TABLE, All Constraints, INSERT, SELECT, WHERE, AND/OR/NOT, ORDER BY, NULL Handling, UPDATE, DELETE, LIMIT, MIN/MAX, COUNT/SUM/AVG, LIKE, IN, BETWEEN, Aliases, All 5 JOIN types, UNION, GROUP BY, HAVING, CASE, EXISTS, ANY/ALL, ALTER TABLE, AUTO_INCREMENT, Comments

Part 2 Advanced:

- 20+ String Functions, Numeric Functions, 20+ Date/Time Functions, All Subquery Types, Window Functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST/LAST VALUE, Running Totals), CTEs (Simple, Multiple, Recursive), Views, Triggers, Stored Procedures, Stored Functions, Transactions & ACID, Indexes & EXPLAIN, Normalization 1NF-BCNF, Error Handling, 15 Optimization Rules, 30 Interview Q&A, 15 Coding Challenges with Multi-Approach Solutions

🗺 Recommended Next Steps

- Load the EduShop schema from Part 1 in MySQL Workbench or DBeaver and run every example yourself
- Solve all 15 coding challenges before looking at the solutions — attempt all approaches
- Practice the 50 interview questions out loud — explaining SQL verbally is the key to interviews
- Build a mini-project: a small inventory/HR system using views, triggers, and stored procedures
- Practice on HackerRank SQL (Easy → Hard) and LeetCode Database section
- Study MySQL documentation: dev.mysql.com/doc/refman/8.0/en/ for function details
- Run EXPLAIN on every query you write at work — build the habit of checking execution plans